

QueueMate

44 课零基础完整学习手册

从项目全景、HTTP、MySQL、Spring Boot、安全与并发，到
Vue3、自动化测试、CI、性能排障和面试表达

使用说明

第一篇按 44 课提供逐课导学：为什么、概念、真实代码、执行流程、常见错误和验收。第二篇完整收录项目现有 11 份专题知识记录，适合在某课后继续深挖。建议一次只学一课，先追代码，再完成验收。

项目基线：Java 21、Spring Boot 3.3.5、MyBatis-Plus 3.5.9、MySQL 8、Vue 3、Vite、Axios、JUnit、Newman、Playwright、JMeter 与 GitHub Actions。

生成日期：2026-09-05。内容以当前工作区代码、README、HANDOFF、学习总纲和 knowledge/01-11 为依据。

目录

- 目录 2
- 第一篇：44 课逐课学习地图 7
- 模块 0：项目全景与学习方法 9
 - 第 0.1 课：用最直白的话讲清 QueueMate 是什么 10
 - 第 0.2 课：认识仓库目录和完整架构 11
- 模块 1：本地运行、网络与一次请求 12
 - 第 1.1 课：进程、端口和 localhost 13
 - 第 1.2 课：HTTP 请求、响应、JSON 和状态码 14
 - 第 1.3 课：从登录按钮追一次完整请求 15
- 模块 2：MySQL 数据模型与 SQL 基础 16
 - 第 2.1 课：表、行、列、主键、外键 17
 - 第 2.2 课：索引、唯一约束、检查约束 18
 - 第 2.3 课：InnoDB、事务和 DECIMAL 19
- 模块 3：Java 与 Spring Boot 分层 20
 - 第 3.1 课：Java 类、对象、接口、枚举和 record 21
 - 第 3.2 课：Spring Bean 与依赖注入 22
 - 第 3.3 课：Controller、Service、Mapper 分层 23
 - 第 3.4 课：参数校验、统一响应和异常处理 24
- 模块 4：登录、JWT、Spring Security 与 RBAC 25
 - 第 4.1 课：注册、密码哈希与登录 26
 - 第 4.2 课：JWT 的结构、签名、过期与局限 27
 - 第 4.3 课：Security 过滤器链和登录态 28
 - 第 4.4 课：RBAC、401/403 与资源归属 29
- 模块 5：核心业务模块与状态机 30
 - 第 5.1 课：商家入驻与多角色账号 31
 - 第 5.2 课：地点与预约时段 32
 - 第 5.3 课：免费预约与取消 33
 - 第 5.4 课：付费预约、钱包和消费码 34
 - 第 5.5 课：排队与繁忙统计 35
- 模块 6：事务、并发与数据一致性 36
 - 第 6.1 课：@Transactional 与回滚边界 37
 - 第 6.2 课：原子 SQL 防预约超卖 38
 - 第 6.3 课：行锁、余额原子扣减与幂等 39
 - 第 6.4 课：并发取号与状态条件更新 40

模块 7: Vue3 前端与响应式页面	41
第 7.1 课: Vue 单文件组件、模板、响应式状态	42
第 7.2 课: Router、路由守卫和多角色导航	43
第 7.3 课: Axios、统一错误处理和前后端联调	44
第 7.4 课: 组件复用、设计 token 与移动端适配	45
模块 8: 自动化测试体系	46
第 8.1 课: 测试金字塔、断言和测试隔离	47
第 8.2 课: JUnit、Mockito 与 Security Test	48
第 8.3 课: Postman/Newman 接口回归	49
第 8.4 课: Playwright 真实浏览器 E2E	50
第 8.5 课: JMeter 并发性能测试	51
模块 9: Git、GitHub Actions 与持续集成	52
第 9.1 课: Git 工作区、暂存区、提交和远程	53
第 9.2 课: GitHub Actions 工作流	54
第 9.3 课: 用真实失败学习 CI 排障	55
模块 10: 性能工程、排障与工程质量	56
第 10.1 课: 性能指标和瓶颈定位	57
第 10.2 课: 系统化排障	58
第 10.3 课: 安全、可维护性和当前边界	59
模块 11: 简历表达与模拟面试	60
第 11.1 课: 项目 30 秒、90 秒、3 分钟介绍	61
第 11.2 课: 简历项目描述与量化结果	62
第 11.3 课: 高频技术追问与压力追问	63
第 11.4 课: 完整模拟面试与查漏补缺	64
第二篇: QueueMate 项目深度知识记录	65
QueueMate 学习记录 01: 项目初始化与本地运行链路	66
1. 本阶段学习目标	66
2. 当前完成情况	66
3. 我的设计过程	67
4. Maven 技术点	68
5. Spring Boot 技术点	69
6. MySQL 数据库设计技术点	70
7. SQL 初始化与模拟数据	71
8. VS Code 开发方式	72
9. 本次遇到的问题与排查方法	72
10. 面向真实业务需要继续加强的部分	73

11. 当前验证结论	74
12. 下一阶段学习任务	74
13. 自测问题	74
QueueMate 学习记录 02: 用户注册、登录与 JWT 鉴权	76
1. 本阶段完成目标	76
2. 代码分层	76
3. 注册设计	76
4. 密码安全	77
5. JWT 设计	78
6. Spring Security 请求过程	79
7. 401 与 403	79
8. 配置与密钥	79
9. 已完成的测试	80
10. 后续安全参考	80
11. 当前边界与后续增强	81
12. 自测问题	81
13. 下一阶段建议	81
QueueMate 学习记录 03: 地点管理与 RBAC	83
1. 本阶段完成目标	83
2. 为什么角色判断和资源归属要分两层	83
3. 创建地点时如何确定所属商家	83
4. 地点名称为什么有两层重复保护	83
5. 地点查询设计	84
6. 参数与状态边界	84
7. 401、403 和资源错误	84
8. 已完成的测试	84
9. Postman 资产	85
10. 下一阶段建议	85
QueueMate 学习记录 04: 固定预约时段	86
1. 本阶段完成目标	86
2. 为什么先做固定时段，再做预约	86
3. 查询规则	86
4. 创建规则	86
5. 重复时段保护	87
6. 时段状态	87
7. ID 精度	87
8. 已完成测试	87

9. 当前边界	88
10. 下一阶段建议	88
QueueMate 免费预约与并发防超卖	89
1. 本轮目标	89
2. 为什么先做免费预约	89
3. 创建预约的事务顺序	89
4. 重复预约的双重保护	90
5. 取消预约的一致性	90
6. 权限和接口边界	90
7. 测试结果	90
8. 下一步	91
QueueMate 付费预约与消费码设计	92
1. 设计结论	92
2. 为什么消费凭证独立建模	92
3. 状态模型	92
4. 收费预约事务	93
5. 消费码生成	93
6. 核销规则	93
7. 取消与退款	94
8. 到期未使用	94
9. 关键测试	94
10. 实现顺序	94
QueueMate 钱包、付费预约与消费码实现	96
1. 本轮完成内容	96
2. 钱包一致性	96
3. 收费预约事务	96
4. 消费码	97
5. 核销与退款竞态	97
6. 权限边界	97
7. 自动化结果	97
8. 真实并发验证	97
9. 当前边界	98
QueueMate 后端收官实现	99
1. 本轮范围	99
2. 并发取号序列	99
3. 有效号码防重	99
4. 状态机	99

5. 自动过期	100
6. 管理员余额调整	100
7. 繁忙时段统计	100
8. 验证结果	100
9. 当前结论	101
Vue3 前端与设计系统实现记录	102
本轮目标	102
技术实现	102
设计决策	102
验证	102
下一步	103
Vue3 全角色前端完成记录	104
1. 本轮目标	104
2. 功能覆盖	104
3. 后端接口边界如何反映在页面	104
4. 设计系统落实	104
5. 验证结果	105
6. 下一步	105
Postman/Newman 可重复全量回归	106
1. 为什么不能直接运行旧集合	106
2. 本轮方案	106
3. 清理端点的安全边界	106
4. 中断与补偿	106
5. Newman 命令	106
6. 首次正式结果	106
附录：关键事实速查	108

第一篇：44 课逐课学习地图

阅读方式

每课的代码入口都使用仓库相对路径。先用编辑器打开对应文件，沿执行流程逐步找方法；遇到不懂的术语，再回到核心概念。不要背完整代码，重点理解输入、规则、状态、SQL 和输出。

课次	主题	模块
0.1	用最直白的话讲清 QueueMate 是什么	项目全景与学习方法
0.2	认识仓库目录和完整架构	项目全景与学习方法
1.1	进程、端口和 localhost	本地运行、网络与一次请求
1.2	HTTP 请求、响应、JSON 和状态码	本地运行、网络与一次请求
1.3	从登录按钮追一次完整请求	本地运行、网络与一次请求
2.1	表、行、列、主键、外键	MySQL 数据模型与 SQL 基础
2.2	索引、唯一约束、检查约束	MySQL 数据模型与 SQL 基础
2.3	InnoDB、事务和 DECIMAL	MySQL 数据模型与 SQL 基础
3.1	Java 类、对象、接口、枚举和 record	Java 与 Spring Boot 分层
3.2	Spring Bean 与依赖注入	Java 与 Spring Boot 分层
3.3	Controller、Service、Mapper 分层	Java 与 Spring Boot 分层
3.4	参数校验、统一响应和异常处理	Java 与 Spring Boot 分层
4.1	注册、密码哈希与登录	登录、JWT、Spring Security 与 RBAC
4.2	JWT 的结构、签名、过期与局限	登录、JWT、Spring Security 与 RBAC
4.3	Security 过滤器链和登录态	登录、JWT、Spring Security 与 RBAC
4.4	RBAC、401/403 与资源归属	登录、JWT、Spring Security 与 RBAC
5.1	商家入驻与多角色账号	核心业务模块与状态机
5.2	地点与预约时段	核心业务模块与状态机
5.3	免费预约与取消	核心业务模块与状态机
5.4	付费预约、钱包和消费码	核心业务模块与状态机
5.5	排队与繁忙统计	核心业务模块与状态机
6.1	@Transactional 与回滚边界	事务、并发与数据一致性
6.2	原子 SQL 防预约超卖	事务、并发与数据一致性
6.3	行锁、余额原子扣减与幂等	事务、并发与数据一致性
6.4	并发取号与状态条件更新	事务、并发与数据一致性
7.1	Vue 单文件组件、模板、响应式状态	Vue3 前端与响应式页面
7.2	Router、路由守卫和多角色导航	Vue3 前端与响应式页面
7.3	Axios、统一错误处理和前后端联调	Vue3 前端与响应式页面
7.4	组件复用、设计 token 与移动端适配	Vue3 前端与响应式页面

课次	主题	模块
8.1	测试金字塔、断言和测试隔离	自动化测试体系
8.2	JUnit、Mockito 与 Security Test	自动化测试体系
8.3	Postman/Newman 接口回归	自动化测试体系
8.4	Playwright 真实浏览器 E2E	自动化测试体系
8.5	JMeter 并发性能测试	自动化测试体系
9.1	Git 工作区、暂存区、提交和远程	Git、GitHub Actions 与持续集成
9.2	GitHub Actions 工作流	Git、GitHub Actions 与持续集成
9.3	用真实失败学习 CI 排障	Git、GitHub Actions 与持续集成
10.1	性能指标和瓶颈定位	性能工程、排障与工程质量
10.2	系统化排障	性能工程、排障与工程质量
10.3	安全、可维护性和当前边界	性能工程、排障与工程质量
11.1	项目 30 秒、90 秒、3 分钟介绍	简历表达与模拟面试
11.2	简历项目描述与量化结果	简历表达与模拟面试
11.3	高频技术追问与压力追问	简历表达与模拟面试
11.4	完整模拟面试与查漏补缺	简历表达与模拟面试

模块 0：项目全景与学习方法

本模块目标

先知道系统解决什么问题、由哪些部分组成，避免一开始就陷入单个类和单条 SQL。

第 0.1 课：用最直白的话讲清 QueueMate 是什么

为什么要学

本课问题

如果连业务对象和角色都没有建立，后面的 JWT、事务和并发只会变成孤立术语。本课先建立‘谁在什么场景下做什么’的最小业务地图。

核心概念（零基础解释）

- 业务链：预约是提前占用未来时段；排队是到店前后领取当天号码，两者互不强制绑定。
- 角色：USER 发起服务，MERCHANT 提供并处理服务，ADMIN 维护平台秩序。
- CRUD：Create、Read、Update、Delete 的缩写；QueueMate 不只是 CRUD，因为它还有权限、状态机、并发和自动化验证。

QueueMate 真实代码入口

- README.md - 项目目标、MVP 功能与非目标
- docs/requirement.md - 角色、范围和验收边界
- frontend/queuemate-web/src/router/index.js - 不同角色可进入的页面

执行流程

1. 访客浏览地点
2. 普通用户登录后预约或取号
3. 商家维护地点、时段并处理号码/消费码
4. 管理员审核商家并执行平台级操作

常见错误与排障

- 把预约和排队说成同一流程
- 把模拟钱包说成真实支付
- 只报技术栈，不说明业务规则

本课验收

- 用 90 秒说清角色、两条业务链和技术价值
- 指出至少两个非目标

第 0.2 课：认识仓库目录和完整架构

为什么要学

本课问题

真实排障必须知道问题可能在页面、接口、业务、SQL、测试还是 CI。目录结构就是系统职责的地图。

核心概念（零基础解释）

- Monorepo：前端、后端、SQL、测试和文档位于同一 Git 仓库。
- 前后端分离：浏览器页面通过 HTTP API 调用后端，而不是由后端直接拼 HTML。
- 调用链：一次操作按时间依次经过的组件集合。

QueueMate 真实代码入口

- frontend/queue-mate-web - Vue 页面与接口调用
- backend/queue-mate-server - Spring Boot API 与业务
- sql - 表结构、模拟数据和迁移
- tests 与 .github/workflows - 本地回归和云端持续集成

执行流程

1. VenueDetailView 收集预约动作
2. api.js/http.js 生成 HTTP 请求
3. Security 与 Controller 接收请求
4. Service 编排规则和事务
5. Mapper 执行 SQL，MySQL 持久化
6. 响应返回页面并显示结果

常见错误与排障

- 把 Controller 当成所有业务逻辑的容器
- 把 Mapper 说成网络请求
- 把页面路由 /bookings 与 API /api/v1/bookings 混淆

本课验收

- 不看资料画出前端到数据库链路
- 为每个顶层目录说出职责

模块 1：本地运行、网络与一次请求

本模块目标

把浏览器、Vite、Spring Boot 与 MySQL 串成可观察的运行链路。

第 1.1 课：进程、端口和 localhost

为什么要学

本课问题

页面打不开时，先判断服务有没有运行和监听，能避免把网络连接失败误判成代码业务错误。

核心概念（零基础解释）

- 进程：程序正在运行的实例；PID 是操作系统分配的进程编号。
- 端口：主机上网络服务的逻辑入口；配置了端口不等于已经监听。
- localhost：发起连接的程序所在机器；手机里的 localhost 指手机自己。

QueueMate 真实代码入口

- frontend/queuemate-web/vite.config.js - Vite 5173 与 /api 代理
- backend/queuemate-server/src/main/resources/application.yml - 后端 8080 与数据库连接
- frontend/queuemate-web/package.json - Vite 启动命令

执行流程

1. 浏览器临时端口连接 Vite 5173
2. Vite 代理连接 Spring Boot 8080
3. Java 通过 JDBC 驱动连接 mysqld 3306

常见错误与排障

- 3306 的监听者是 mysqld，不是 SQL
- 5173 被占用时 Vite 可能回退到其他端口
- 0.0.0.0 是监听范围，不是通常的访问目标

本课验收

- 解释 5173、8080、3306 分别由谁监听
- 区分 ERR_CONNECTION_REFUSED 与 HTTP 500

第 1.2 课：HTTP 请求、响应、JSON 和状态码

为什么要学

本课问题

前后端联调的证据都在请求方法、地址、头、请求体、状态码和响应体中。

核心概念（零基础解释）

- HTTP 请求/响应：客户端发出方法、地址、头和可选正文；服务器返回状态、头和正文。
- JSON：使用对象、数组、字符串、数字、布尔值和 null 表达结构化数据。
- 状态码与业务码：HTTP 码描述通用结果类别，ApiResponse.code 描述 QueueMate 的具体业务原因。

QueueMate 真实代码入口

- frontend/queuebate-web/src/services/http.js - baseUrl、JWT 与错误归一化
- frontend/queuebate-web/src/services/api.js - GET/POST/PATCH 方法
- common/api/ApiResponse.java 与 common/exception/GlobalExceptionHandler.java

执行流程

1. POST /api/v1/bookings 携带 JSON 与 Bearer token
2. Security 验证身份和角色
3. Controller 反序列化并校验
4. Service 成功或抛出业务异常
5. 成功返回 201/0；冲突返回 409/具体业务码
6. Axios 成功提取 data，失败生成统一 Error

常见错误与排障

- 把 HTTP 201 与业务码 0 当成同一个值
- 没有 JWT 时先得到 401，参数校验尚未运行
- 网络失败没有 HTTP 响应，前端 status 为 0

本课验收

- 准确解释 200/201/400/401/403/404/409/500
- 用 Network 面板拆解一次请求

第 1.3 课：从登录按钮追一次完整请求

为什么要学

本课问题

登录横跨表单、HTTP、密码校验、JWT、数据库与前端会话，是理解全栈链路的最佳样本。

核心概念（零基础解释）

- 序列化/反序列化：Java 对象与 JSON 文本之间的转换。
- 密码哈希校验：对输入密码执行同类算法，与数据库哈希比较，不解密原密码。
- 会话状态：当前前端把 token 和用户信息放入 localStorage，并通过响应式状态使用。

QueueMate 真实代码入口

- views/LoginView.vue
- services/api.js 与 services/http.js
- AuthController.java -> AuthService.java -> UserMapper.java
- JwtTokenService.java 与 state/auth.js

执行流程

1. 提交用户名密码
2. POST /auth/login
3. AuthService 查询用户并校验状态/密码
4. JwtTokenService 签发 token
5. LoginResponse 返回 token 与 user
6. authState.setSession 保存并跳转角色首页

常见错误与排障

- 认为 JWT 是每次请求重新登录取得
- 只检查前端登录成功提示，不看 Network
- 登录失败时向外暴露 ‘用户名不存在’ 或 ‘密码错误’ 的差异

本课验收

- 从 LoginView 逐文件追到 UserMapper
- 说明登录 401 与受保护接口 401 的共同点和差异

模块 2：MySQL 数据模型与 SQL 基础

本模块目标

理解数据怎样落表，以及数据库约束为什么是并发情况下的最后防线。

第 2.1 课：表、行、列、主键、外键

为什么要学

本课问题

Service 最终操作的不是抽象业务，而是一行行有关系的数据；先懂关系模型才能追 SQL。

核心概念（零基础解释）

- 表/行/列：表描述一类实体，行是一条记录，列是记录的属性。
- 主键：唯一标识一行；QueueMate 多数表使用 BIGINT id。
- 外键：保证引用的父记录存在，保护表之间的引用完整性。

QueueMate 真实代码入口

- sql/schema.sql - 11 张表
- docs/db_design.md - 关系说明
- 各模块 Entity 类 - Java 字段到表列的映射

执行流程

1. users 与 user_roles 描述账号及多角色
2. venues 属于商家
3. booking_slots 属于地点
4. bookings 关联用户、地点和时段
5. wallets/wallet_transactions 记录余额
6. booking_vouchers、queue_tickets 与 queue_daily_sequences 支撑履约和排队

常见错误与排障

- 把主键等同于业务编号 booking_no
- 删除父记录前忽略外键顺序
- 把 Entity 当成可以直接接收所有外部请求的 DTO

本课验收

- 说出 11 张表的职责
- 从 booking.id 关联到用户、地点、时段和消费码

第 2.2 课：索引、唯一约束、检查约束

为什么要学

本课问题

代码预检查在并发下可能同时通过，数据库约束负责最后一刻拒绝非法数据。

核心概念（零基础解释）

- 普通索引：用额外存储换取查询速度，不自动阻止重复。
- 唯一约束：禁止一组列出现重复组合。
- CHECK 约束：要求每行数据满足布尔条件，例如 reserved_count 不得超出 capacity。

QueueMate 真实代码入口

- schema.sql 中 users.username、消费码、业务流水唯一键
- bookings 的 active_slot_id 生成列与唯一键
- booking_slots 的 reserved_count CHECK

执行流程

1. Service 先检查并提供友好消息
2. 并发请求可能在检查后同时写入
3. 唯一键/CHECK 在数据库写入点拒绝非法结果
4. 后端捕获 DuplicateKeyException 转成稳定业务码

常见错误与排障

- 认为有索引就不会重复
- 只依赖前端校验
- 把所有唯一键冲突原样返回数据库异常

本课验收

- 用重复预约解释双层保护
- 区分性能索引与一致性约束

第 2.3 课：InnoDB、事务和 DECIMAL

为什么要学

本课问题

付费预约同时影响容量、余额、预约、流水和消费码，任何一步失败都不能留下半套数据。

核心概念（零基础解释）

- InnoDB：MySQL 存储引擎，支持事务、行锁、崩溃恢复和外键。
- 事务：一组操作一起提交或一起回滚；原子性只保证同一事务边界内的操作。
- DECIMAL：十进制定点数，适合金额；float/double 的二进制表示可能产生精度误差。

QueueMate 真实代码入口

- schema.sql 的 engine=InnoDB 与 DECIMAL 字段
- BookingService.create/cancel 的 @Transactional
- WalletService 的余额和流水更新

执行流程

1. 开始事务
2. 占用名额并可能扣款
3. 写预约、流水和消费码
4. 全部成功提交
5. 任一步抛出运行时异常则整体回滚

常见错误与排障

- 把回滚说成额外退款
- 用 double 保存金额
- 误以为跨网络系统的所有操作自动加入本地数据库事务

本课验收

- 解释扣款成功但预约写入失败时的结果
- 说明 wallets 与流水为何必须同事务

模块 3：Java 与 Spring Boot 分层

本模块目标

掌握对象、依赖注入和 Controller-Service-Mapper 的职责边界。

第 3.1 课：Java 类、对象、接口、枚举和 record

为什么要学

本课问题

QueueMate 源码大量使用 Entity、Request、Response、Mapper、Enum 和 record；分不清类型就无法阅读方法签名。

核心概念（零基础解释）

- 类与对象：类是结构和行为的定义，对象是运行时实例。
- 接口：声明能力和方法契约；MyBatis Mapper 接口由框架生成实现。
- 枚举：有限状态集合；record：适合不可变数据载体，自动生成构造器和访问方法。

QueueMate 真实代码入口

- booking/Booking.java - Entity 类
- BookingStatus.java - 枚举
- BookingCreateRequest.java 与 BookingResponse.java - record DTO
- BookingMapper.java - 接口

执行流程

1. JSON 转 Request record
2. Service 创建/读取 Entity 对象
3. Mapper 接口持久化 Entity
4. Entity 转 Response record
5. Jackson 把 Response 转 JSON

常见错误与排障

- 把类名和对象变量名混为一谈
- 认为接口不能运行所以 Mapper 没实现
- 直接用 Entity 接外部请求导致可写字段失控

本课验收

- 任选一个包标注 Entity/Request/Response/Mapper/Enum
- 解释 record 适合 DTO 的原因

第 3.2 课：Spring Bean 与依赖注入

为什么要学

本课问题

对象如果各自 new 依赖，配置、测试替换、事务代理和生命周期会难以统一。

核心概念（零基础解释）

- Bean：由 Spring 容器创建和管理的对象。
- 依赖注入：对象声明自己需要什么，由容器提供实例。
- 构造器注入：依赖通过构造器传入，便于保持必需依赖和单元测试替换。

QueueMate 真实代码入口

- @Service BookingService
- @RestController BookingController
- @Mapper BookingMapper
- BookingController(BookingService) 与 BookingService(...) 构造器

执行流程

1. 启动时组件扫描发现注解
2. Spring 创建 Mapper 代理
3. 创建 Service 并注入 Mapper
4. 创建 Controller 并注入 Service
5. 请求到达时复用这些 Bean

常见错误与排障

- 在 Service 内 new Mapper
- 字段注入导致依赖隐藏
- 忘记组件扫描范围或注解，启动时报找不到 Bean

本课验收

- 解释 Service 为什么不自己 new Mapper
- 从构造器列出 BookingService 的五个依赖

第 3.3 课：Controller、Service、Mapper 分层

为什么要学

本课问题

分层让 HTTP、业务规则和 SQL 各自可读、可测、可替换，避免一个方法承担全部责任。

核心概念（零基础解释）

- Controller：处理 Web 契约，例如路径、方法、参数和响应。
- Service：组织业务规则、状态变化、事务和跨模块调用。
- Mapper：把 Java 调用转换为数据库查询与更新。

QueueMate 真实代码入口

- VenueController.java
- VenueService.java
- VenueMapper.java
- VenueCreateRequest.java 与 VenueResponse.java

执行流程

1. GET /venues/{id} 匹配 Controller
2. Controller 调 Service.get
3. Service 校验资源存在/规则
4. Mapper 读取 venues
5. Service 转换 Response
6. Controller 用 ApiResponse 包装

常见错误与排障

- Controller 堆积复杂业务
- Mapper 决定完整流程
- 前端隐藏按钮代替后端权限

本课验收

- 追踪地点查询并标注每层输入输出
- 给新需求判断应修改哪一层

第 3.4 课：参数校验、统一响应和异常处理

为什么要学

本课问题

客户端需要稳定的成功结构和可预测错误，而不是每个接口返回不同形状或暴露堆栈。

核心概念（零基础解释）

- Jakarta Validation：用 @NotNull、@Positive、@Size 等注解描述输入约束。
- 统一响应：ApiResponse 固定 code、message、data。
- 全局异常处理：@RestControllerAdvice 将异常集中转换为 HTTP 状态与 JSON。

QueueMate 真实代码入口

- BookingCreateRequest.java
- ApiResponse.java
- BusinessException.java
- GlobalExceptionHandler.java

执行流程

1. 请求体反序列化
2. @Valid 执行字段校验
3. 参数异常转 400/PARAM_INVALID
4. 业务异常携带 HttpStatus 和 code
5. 未知异常记录日志并返回 500/SYSTEM_ERROR

常见错误与排障

- 把 400 当成 Service 返回
- 把所有异常吞掉并返回 200
- 把数据库异常或堆栈直接暴露给前端

本课验收

- 预测 slotId=-3 的结束位置
- 解释 HTTP 状态码和业务码的分工

模块 4：登录、JWT、Spring Security 与 RBAC

本模块目标

弄清身份、角色和资源归属三道安全检查。

第 4.1 课：注册、密码哈希与登录

为什么要学

本课问题

账号系统必须既保护密码，又保证用户和钱包初始化一致，并避免通过错误消息泄露账号是否存在。

核心概念（零基础解释）

- 哈希：单向摘要；密码验证是重新计算并比较，不是解密。
- 盐：让相同密码产生不同哈希，BCrypt 会管理盐和本地。
- 账户枚举：攻击者通过差异化错误判断用户名是否存在。

QueueMate 真实代码入口

- AuthController.java
- AuthService.java
- RegisterRequest/LoginRequest
- users、wallets 与 username 唯一键

执行流程

1. 注册参数校验
2. BCrypt 编码密码
3. 同事务插入 USER/ACTIVE 用户与零余额钱包
4. 登录查询用户并检查状态
5. matches 校验密码
6. 签发 JWT

常见错误与排障

- 保存或记录明文密码
- 注册只写用户不写钱包
- 登录分别返回 ‘用户不存在’ 和 ‘密码错误’

本课验收

- 解释数据库为何无法恢复原密码
- 说明注册事务失败时钱包和用户的结果

第 4.2 课：JWT 的结构、签名、过期与局限

为什么要学

本课问题

JWT 让后端无需服务器 Session 也能识别请求，但签名不等于加密，过期和撤销也有明确边界。

核心概念（零基础解释）

- JWT：由 header、payload、signature 三段组成的令牌。
- 签名：防止内容被篡改，不负责隐藏 payload。
- 无状态：服务端不为每个登录保存 Session；每次请求携带 token。

QueueMate 真实代码入口

- JwtTokenService.java
- JwtProperties.java
- application.yml 的 JWT 配置
- LoginResponse.java

执行流程

1. 登录时用密钥签发
2. payload 保存用户标识和角色等声明
3. 设置过期时间
4. 请求时解析并验证签名/过期
5. 当前项目再查数据库刷新实际用户状态

常见错误与排障

- 把 JWT 说成加密用户数据
- 把敏感密码放进 payload
- 忽略当前项目没有 Refresh Token、黑名单和密钥轮换

本课验收

- 解释三段各做什么
- 说明 token 泄露后的风险和当前边界

第 4.3 课：Security 过滤器链和登录态

为什么要学

本课问题

Controller 之前已经发生 JWT 解析和身份建立；不了解过滤器顺序就会误判为什么代码没有进入断点。

核心概念（零基础解释）

- 过滤器链：请求进入 Controller 前后依次执行的一组 Web 过滤器。
- SecurityContext：保存本次请求已认证身份的上下文。
- AuthenticationPrincipal：Controller 从安全上下文取得当前用户。

QueueMate 真实代码入口

- SecurityConfig.java
- JwtAuthenticationFilter.java
- RestAuthenticationEntryPoint.java
- AuthenticatedUser.java

执行流程

1. 读取 Authorization
2. 确认 Bearer 前缀
3. 解析 JWT
4. 查询实际用户和角色
5. 构造 Authentication 放入 SecurityContext
6. 权限通过后进入 Controller

常见错误与排障

- Bearer 本身被当成 token
- token 无效仍继续当匿名用户执行写接口
- 401 时不清理前端过期会话

本课验收

- 画出 Authorization 到 principal 的链路
- 说明公开接口为何可跳过登录要求

第 4.4 课：RBAC、401/403 与资源归属

为什么要学

本课问题

有 MERCHANT 角色只说明是某个商家，不能自动获得修改所有商家地点的权力。

核心概念（零基础解释）

- RBAC：Role-Based Access Control，按角色授予能力。
- 资源归属：进一步检查目标数据是否属于当前账号。
- 最小权限：只授予完成职责所需的权限范围。

QueueMate 真实代码入口

- @PreAuthorize 注解
- VenueService.requireOwnerOrAdmin
- SecurityConfig 的公开/受保护路径
- 前端 router meta.roles

执行流程

1. 无身份 -> 401
2. 身份有效但角色不符 -> 403
3. 角色允许后加载目标地点
4. MERCHANT 校验 merchant_id 是否等于当前用户
5. ADMIN 按规则可越过归属检查

常见错误与排障

- 只在前端隐藏按钮
- 把角色权限与数据归属混成一次判断
- 为防信息泄露随意把 403 改成 404 而不统一策略

本课验收

- 解释商家修改别人地点为何是 403
- 区分角色检查和所有者检查

模块 5：核心业务模块与状态机

本模块目标

沿着商家、地点、预约、钱包、消费码与排队理解真实业务规则。

第 5.1 课：商家入驻与多角色账号

为什么要学

本课问题

同一账号可能既是顾客又是商家，角色不是只能保存在 users.role 的单值标签。

核心概念（零基础解释）

- 多对多关系：一个用户有多个角色，一个角色属于多个用户。
- 审核状态机：PENDING 只能转 APPROVED 或 REJECTED。
- 悲观锁：审核时锁住申请行，阻止并发重复处理。

QueueMate 真实代码入口

- user_roles 与 merchant_applications 表
- MerchantApplicationService/Mapper
- MerchantApplicationView.vue 与 AdminMerchantApplicationsView.vue

执行流程

1. USER 提交申请
2. 唯一生成列阻止同一人多个待审申请
3. ADMIN 锁定并审核
4. 通过后 insert ignore 授予 MERCHANT
5. 用户刷新后可切换工作区

常见错误与排障

- 审核通过后删除 USER 角色
- 重复审批同一申请
- 误以为通过审核就自动创建并发布门店

本课验收

- 画出 PENDING 状态转换
- 说明 user_roles 解决什么问题

第 5.2 课：地点与预约时段

为什么要学

本课问题

预约不是直接对地点计数，而是对地点下某一天、某段时间、某个容量的 slot 占位。

核心概念（零基础解释）

- Venue：提供服务的地点，保存归属、启停和是否支持预约/排队。
- BookingSlot：具体日期、开始/结束时间、容量、价格和开放状态。
- 聚合关系：一个地点包含多个时段，时段不能脱离地点规则。

QueueMate 真实代码入口

- VenueService/Controller
- BookingSlotService/Controller
- booking_slots 表
- OperatorVenueView.vue

执行流程

1. 商家创建地点
2. 启用 bookingEnabled
3. 创建未来时段并校验时间/容量/价格
4. 公开查询只返回符合日期和状态条件的时段
5. 商家可 OPEN/CLOSED

常见错误与排障

- 只校验 slot 状态不校验 venue 状态
- 首版只防完全相同时段，不防部分重叠
- 创建后当前只支持开关，不支持编辑核心字段

本课验收

- 区分 venueId、slotId、bookingId
- 说明地点停用对预约的影响

第 5.3 课：免费预约与取消

为什么要学

本课问题

免费预约已经包含重复保护、容量竞争、状态更新和取消回补，是理解核心一致性的最小闭环。

核心概念（零基础解释）

- 有效预约：BOOKED 状态占用名额；取消历史保留但不继续占位。
- 条件更新：把前置状态写进 WHERE，只有状态仍符合时才更新。
- 取消回补：预约成功占用一个名额，合法取消必须释放一个。

QueueMate 真实代码入口

- BookingService.create/cancel
- BookingSlotMapper.reserveCapacity/releaseCapacity
- BookingMapper 条件更新
- BookingsView.vue

执行流程

1. 检查用户/时段/地点/重复
2. 原子占用容量
3. 保存 BOOKED/NOT_REQUIRED 预约
4. 取消时条件改为 CANCELLED
5. reserved_count 原子减一
6. 取消后允许重新预约

常见错误与排障

- 先插预约再占容量而不设事务
- 重复取消导致多次释放
- 删除预约历史来允许重新预约

本课验收

- 画出 BOOKED -> CANCELLED
- 解释取消两次为何一次成功一次 409

第 5.4 课：付费预约、钱包和消费码

为什么要学

本课问题

付费预约把容量、资金记录和履约凭证绑在一个事务中，是系统最复杂的业务链。

核心概念（零基础解释）

- 钱包快照与流水：balance 方便快速读取，transaction 解释每次变化。
- 消费凭证：把 ‘已付款预约’ 与 ‘到店履约’ 独立建模。
- 业务唯一键：biz_type + biz_no + type 阻止同一业务重复扣款或退款。

QueueMate 真实代码入口

- WalletService/Mapper
- BookingVoucherService/Mapper
- BookingService.create/cancel
- wallet_transactions 与 booking_vouchers 表

执行流程

1. 占名额
2. 锁钱包并检查余额
3. 条件扣款并写 PAY 流水
4. 保存 PAID 预约
5. 生成 AVAILABLE 消费码
6. 取消则退款并将凭证置 VOID；核销则预约 FULFILLED

常见错误与排障

- 把钱包当真实支付平台
- 只更新余额不写流水
- 核销和退款不加锁导致同一凭证走向两个终态

本课验收

- 追踪余额 50 -> 30 -> 50
- 解释为什么消费码独立建表

第 5.5 课：排队与繁忙统计

为什么要学

本课问题

排队是按地点和日期生成号码并推进状态；统计则把预约与排队事件聚合成可读趋势。

核心概念（零基础解释）

- 排队状态机：WAITING -> CALLED -> COMPLETED/MISSED。
- 每日序列：每个地点每天独立维护 last_no。
- 聚合查询：按小时分组并计算预约数、取号数或热度分数。

QueueMate 真实代码入口

- QueueTicketService/Mapper
- QueueSequenceMapper
- BusyHoursService/Mapper
- QueueView.vue 与 OperatorVenueView.vue

执行流程

1. 用户取号获得 WAITING
2. 商家叫号变 CALLED
3. 服务完成或过号进入终态
4. 统计 SQL 按地点/日期/小时合并预约与排队数量

常见错误与排障

- 允许 WAITING 直接 COMPLETED
- 全平台共用一个号码序列
- 把聚合统计当作实时排队真相

本课验收

- 画出合法/非法状态转换
- 解释号码为何按地点和日期隔离

模块 6：事务、并发与数据一致性

本模块目标

解释为什么并发请求不会超卖、重复扣款或生成重复号码。

第 6.1 课：@Transactional 与回滚边界

为什么要学

本课问题

多个数据库写操作只有在同一事务中，才能避免名额已占、钱已扣但预约不存在。

核心概念（零基础解释）

- 事务边界：从带 @Transactional 的公开方法进入，到正常返回提交或异常退出回滚。
- 传播与代理：Spring 通常通过代理开启事务；同类内部自调用可能绕过代理。
- 回滚规则：默认 RuntimeException/Error 触发回滚，受检异常需按配置处理。

QueueMate 真实代码入口

- BookingService.create/cancel
- WalletService 的写方法
- MerchantApplicationService 审核
- 测试中的异常回滚场景

执行流程

1. 代理开启事务
2. 多个 Mapper 共用当前连接/事务
3. 正常返回 commit
4. 业务或运行时异常 rollback
5. 调用者收到错误响应

常见错误与排障

- 捕获异常后不再抛出导致事务提交
- 私有方法加 @Transactional 期待生效
- 把外部 HTTP 调用误认为自动参与 MySQL 事务

本课验收

- 指出付费预约事务内的所有写操作
- 解释回滚与补偿的区别

第 6.2 课：原子 SQL 防预约超卖

为什么要学

本课问题

多个请求可能同时读到最后一个名额；必须让数据库在一次更新中同时检查和加一。

核心概念（零基础解释）

- 竞态条件：结果依赖多个线程不可预测的执行时序。
- 原子操作：对外表现为不可分割的检查与更新。
- 受影响行数：条件 UPDATE 返回 1 表示成功，0 表示条件已不满足。

QueueMate 真实代码入口

- BookingSlotMapper.reserveCapacity
- BookingService.throwCapacityFailure
- tests/jmeter/concurrent-booking.jmx

执行流程

1. 12 个用户同时提交
2. 每条 UPDATE 要求 reserved_count < capacity
3. MySQL 串行处理冲突写入
4. 只有前三条更新 1 行
5. 其余返回 0 并被解释为 BOOKING_SLOT_FULL

常见错误与排障

- 先 SELECT 剩余名额再普通 +1
- 把前端按钮禁用当并发保护
- 只看 JMeter 进程码不核对业务成功数

本课验收

- 手画两个线程抢最后一位的错误时序
- 解释 3 成功/9 冲突为何是正确结果

第 6.3 课：行锁、余额原子扣减与幂等

为什么要学

本课问题

同一钱包或消费码被并发操作时，需要锁、条件更新和唯一键共同保证只生效一次。

核心概念（零基础解释）

- SELECT ... FOR UPDATE：事务内读取并锁定目标行，其他写事务等待。
- 条件扣减：UPDATE 同时要求 `balance >= amount`，受影响 0 行表示余额不足或状态不符。
- 幂等：同一个业务请求重复到达时，不产生重复资金效果。

QueueMate 真实代码入口

- `WalletMapper.selectByUserIdForUpdate/deductBalance`
- `WalletService`
- `wallet_transactions` 唯一键
- `BookingVoucherMapper` 的 FOR UPDATE

执行流程

1. 锁钱包得到稳定 `balanceBefore`
2. 条件扣减
3. 写带业务键的流水
4. 提交后释放锁
5. 重复请求被状态或唯一约束拒绝

常见错误与排障

- 只有 Java `synchronized`，无法覆盖多进程
- 只有行锁，没有业务唯一键
- 把接口返回相同理解为幂等的唯一定义

本课验收

- 解释锁与唯一键各防什么
- 说明余额不足为何用条件 SQL 再兜底

第 6.4 课：并发取号与状态条件更新

为什么要学

本课问题

并发取号既要号码连续不重复，后续叫号也要防止两个操作者同时推进同一号码。

核心概念（零基础解释）

- Upsert：不存在则插入、存在则更新。
- LAST_INSERT_ID 技巧：在当前数据库连接中取回本次生成或更新的号码。
- Expected-to-target 更新：WHERE status=expected, SET status=target。

QueueMate 真实代码入口

- QueueSequenceMapper.next/currentConnectionValue
- queue_daily_sequences 唯一键
- QueueTicketMapper 状态更新
- QueueTicketService

执行流程

1. 按 venue_id + date upsert 序列
2. 原子 last_no+1 并从同一连接读取
3. 插入 WAITING ticket
4. 叫号用 WAITING -> CALLED 条件更新
5. 并发第二次更新因状态变化返回 0

常见错误与排障

- 先查最大号码再 +1
- next 与 currentConnectionValue 不在同一连接
- 无 expected status 的无条件覆盖

本课验收

- 解释 6 个并发用户为何得到连续唯一号码
- 预测同一票并发叫号结果

模块 7：Vue3 前端与响应式页面

本模块目标

理解页面状态、路由、Axios 与响应式布局如何共同工作。

第 7.1 课：Vue 单文件组件、模板、响应式状态

为什么要学

本课问题

页面不是静态 HTML；数据变化后 Vue 要自动重新渲染 loading、列表、错误和按钮状态。

核心概念（零基础解释）

- 单文件组件：.vue 文件把 script、template、style 组织在一起。
- ref/reactive：让值或对象成为可追踪的响应式状态。
- computed：根据其他响应式值派生结果并缓存依赖关系。

QueueMate 真实代码入口

- VenueDetailView.vue
- WalletView.vue
- components/StatePanel.vue
- components/VenueCard.vue

执行流程

1. 组件创建响应式状态
2. onMounted 请求数据
3. 赋值触发依赖更新
4. template 根据 v-if/v-for 渲染
5. 点击 @click 执行异步函数
6. finally 恢复 loading/action 状态

常见错误与排障

- 忘记 ref 在 script 中使用 .value
- 直接修改非响应式普通变量期待页面刷新
- 异步失败不在 finally 恢复 loading

本课验收

- 在一个 View 中找出 ref/computed/event
- 解释四态 loading/empty/error/success

第 7.2 课：Router、路由守卫和多角色导航

为什么要学

本课问题

单页应用需要把 URL 映射到组件，并在进入页面前处理登录要求和角色工作区。

核心概念（零基础解释）

- SPA：单页应用在不整页刷新时切换组件视图。
- 路由守卫：导航发生前执行的检查，可重定向但不是后端安全边界。
- 懒加载：访问路由时再加载对应页面代码。

QueueMate 真实代码入口

- router/index.js
- state/auth.js
- components/AppShell.vue
- LoginView.vue

执行流程

1. 匹配 path 到 component
2. 读取 meta.requiresAuth/roles
3. 未登录跳 /login?redirect=原地址
4. 角色不符跳对应首页
5. 登录成功回到 redirect
6. 多角色账号切换 activeRole

常见错误与排障

- 把路由守卫当作后端鉴权
- 重定向丢失原目标
- 只保存单一 role 导致多身份失真

本课验收

- 追踪未登录访问 /wallet
- 解释 ADMIN/MERCHANT 的默认首页

第 7.3 课：Axios、统一错误处理和前后端联调

为什么要学

本课问题

每个页面都手写 token、baseUrl 和错误判断会重复且不一致，Axios 实例负责集中处理。

核心概念（零基础解释）

- Axios 实例：保存 baseUrl、超时和默认头的 HTTP 客户端。
- 拦截器：请求发出前或响应交给调用者前统一加工。
- 同源代理：开发时浏览器请求 5173 的 /api，由 Vite 转到 8080。

QueueMate 真实代码入口

- services/http.js
- services/api.js
- vite.config.js
- 各 View 的 try/catch/finally

执行流程

1. api.js 描述资源方法
2. 请求拦截器注入 Bearer token
3. Vite 代理
4. 成功拦截器取 response.data.data
5. 失败拦截器生成 code/status/message
6. 401 清会话并带 redirect 登录

常见错误与排障

- 页面再次读取 result.data
- 把 409 当网络错误
- VITE_API_BASE_URL 覆盖后遗漏 /api/v1

本课验收

- 解释组件最终拿到哪一层 data
- 按 Network -> 后端日志 -> SQL 顺序排一次 401/409

第 7.4 课：组件复用、设计 token 与移动端适配

为什么要学

本课问题

统一组件和 token 能让视觉、状态和可访问性规则集中维护，而不是复制大量略有不同的 CSS。

核心概念（零基础解释）

- 组件复用：通过 props、slots 和 events 组合通用 UI。
- 设计 token：把颜色、间距、字号等设计决策命名为变量。
- 响应式布局：根据可用宽度调整网格、按钮和信息层级。

QueueMate 真实代码入口

- components/AppShell/StatePanel/VenueCard/StickerBadge.vue
- styles/tokens.css 与 global.css
- design-system/MASTER.md

执行流程

1. token 定义语义颜色和尺寸
2. 共享组件引用 token
3. 页面组合组件
4. 媒体查询在 760px 等断点重排
5. 真实浏览器检查 1440/375/横屏/reduced-motion

常见错误与排障

- 每页硬编码不同颜色
- 只缩小字体不重排布局
- 装饰元素承载唯一状态信息
- 触摸目标低于 44px

本课验收

- 从 token 追到一个组件样式
- 解释移动端横向溢出的排查方法

模块 8：自动化测试体系

本模块目标

区分单元、接口、UI 和性能测试的职责，并理解测试隔离。

第 8.1 课：测试金字塔、断言和测试隔离

为什么要学

本课问题

一种测试无法覆盖所有风险；快速的小测试与少量真实端到端测试要形成互补。

核心概念（零基础解释）

- 测试金字塔：底层单元测试多而快，上层端到端测试少而真实。
- 断言：把预期写成可自动判断的条件。
- 测试隔离：每次运行拥有独立数据，不依赖上次残留或修改真实数据。

QueueMate 真实代码入口

- backend/src/test - 144 个后端测试
- tests/postman、tests/playwright、tests/jmeter
- docs/test_plan.md

执行流程

1. 单元测试验证局部规则
2. Controller 测试验证 HTTP/权限
3. Newman 验证完整 API
4. Playwright 验证浏览器流程
5. JMeter 验证并发不变量
6. CI 统一运行并保存报告

常见错误与排障

- 只追求测试数量
- 用同一固定用户名导致重复运行失败
- 断言只看 HTTP 200 不看业务数据

本课验收

- 为一个缺陷选择最合适测试层
- 解释 runId 与清理的价值

第 8.2 课：JUnit、Mockito 与 Security Test

为什么要学

本课问题

后端规则需要快速、可重复验证，同时区分 ‘业务代码错’ 和 ‘HTTP/权限映射错’ 。

核心概念（零基础解释）

- JUnit：组织测试用例、生命周期和断言。
- Mockito：用可控替身代替依赖，验证调用与异常分支。
- MockMvc/Security Test：不启动真实浏览器也能测试 Controller、状态码和角色。

QueueMate 真实代码入口

- BookingServiceTest.java
- BookingControllerSecurityTest.java
- VenueControllerSecurityTest.java
- pom.xml 测试依赖

执行流程

1. Arrange 准备对象和 mock
2. Act 调用方法或 MockMvc 请求
3. Assert 检查返回/异常/调用
4. Security Test 构造 USER/MERCHANT/ADMIN 身份
5. Surefire 汇总结果

常见错误与排障

- mock 掉被测对象本身
- 只验证没有异常
- Service 单测误以为已经验证真实 SQL

本课验收

- 区分 ServiceTest 与 ControllerSecurityTest
- 为余额不足设计 Arrange/Act/Assert

第 8.3 课：Postman/Newman 接口回归

为什么要学

本课问题

完整 API 回归需要真实 Spring Boot 和 MySQL，并可在命令行及 CI 中重复运行。

核心概念（零基础解释）

- Collection：按顺序组织请求、脚本和断言。
- Environment：保存 baseUrl、token、动态 ID 等运行变量。
- Newman：Postman 集合的命令行执行器。

QueueMate 真实代码入口

- tests/postman/QueueMate.postman_collection.json
- QueueMate.local.postman_environment.json
- package.json
- 11-postman-newman-repeatable-regression.md

执行流程

1. 生成唯一 runId
2. 注册/登录并保存 token
3. 创建地点、时段、预约等动态数据
4. 逐接口断言
5. 最后 ADMIN 调清理端点
6. 检查 remainingArtifacts=0 和 JSON 报告

常见错误与排障

- 使用 --bail 导致清理不执行
- 普通 profile 下调用清理端点
- 集合 JSON 可解析就声称回归通过

本课验收

- 解释 45 请求/99 断言的证据含义
- 说明中断后如何补偿清理

第 8.4 课：Playwright 真实浏览器 E2E

为什么要学

本课问题

接口正确不代表表单、路由、提示和多角色浏览器操作能够组成可用流程。

核心概念（零基础解释）

- E2E：从用户界面入口到真实后端和数据库的端到端验证。
- Browser Context：彼此隔离的浏览器会话，可同时模拟用户和商家。
- Trace/截图/视频：失败时保留页面、网络和操作证据。

QueueMate 真实代码入口

- tests/playwright/playwright.config.js
- support/api-fixture.js
- support/ui-actions.js
- specs 下 6 条核心链路

执行流程

1. API fixture 创建独立数据
2. 浏览器执行真实登录和点击
3. 多 context 模拟角色协作
4. 断言 UI 与后端状态
5. finally 调清理端点
6. 失败保留 trace/screenshots/video

常见错误与排障

- 固定 sleep 代替等待可观察状态
- 多个角色共用同一 context
- 只测 happy path 不验最终业务状态

本课验收

- 说明为何 E2E 数量少于单测
- 从失败 trace 判断页面、网络还是接口问题

第 8.5 课：JMeter 并发性能测试

为什么要学

本课问题

防超卖必须通过真正同时发生的请求验证，串行调用无法暴露竞态。

核心概念（零基础解释）

- 线程组：配置并发虚拟用户与执行步骤。
- 同步定时器：让多个线程在同一屏障后同时发送关键请求。
- JTL 与百分位：JTL 保存每个采样；P90/P95 描述 90%/95% 请求不超过的时间。

QueueMate 真实代码入口

- tests/jmeter/concurrent-booking.jmx
- run.ps1
- write-summary-zh.ps1
- README.md

执行流程

1. 创建容量 3 的时段和 12 用户
2. 各用户独立登录
3. 同步提交预约
4. 断言 3 个 201 与 9 个 409/FULL
5. 查询 reservedCount=3
6. 扫描 JTL 失败采样并清理

常见错误与排障

- JMeter 进程返回 0 就认为业务通过
- 只看平均值忽略尾延迟
- 测试后残留 12 个用户和预约

本课验收

- 解释 56 个采样与 12 个预约请求的关系
- 区分平均值、P90、P95、错误率

模块 9：Git、GitHub Actions 与持续集成

本模块目标

把本地修改变成可追踪提交，并让云端自动重复验证。

第 9.1 课：Git 工作区、暂存区、提交和远程

为什么要学

本课问题

版本控制既是回退保险，也是多人协作边界；错误的回退命令可能覆盖用户已有修改。

核心概念（零基础解释）

- 工作区：当前磁盘文件；暂存区：下一次提交的候选快照；提交：带父关系和说明的不可变快照。
- 分支：指向提交的可移动引用。
- 远程：GitHub 等共享仓库；push 上传本地提交，fetch 获取远程引用。

QueueMate 真实代码入口

- .git 仓库元数据
- .gitignore
- HANDOFF.md 的安全流程
- README 中验证证据

执行流程

1. git status 看范围
2. git diff 审查未暂存修改
3. git add 精确暂存
4. git diff --cached 复核
5. git commit
6. git push 并核对远程

常见错误与排障

- 未经确认 reset --hard
- 把构建产物和密钥提交
- 混入无关修改或覆盖他人工作

本课验收

- 解释三棵树模型
- 给出安全提交前检查清单

第 9.2 课：GitHub Actions 工作流

为什么要学

本课问题

CI 把 ‘我本机能跑’ 升级为每次代码变化都在干净环境自动验证。

核心概念（零基础解释）

- Workflow：一个 YAML 自动化流程；trigger 决定何时执行。
- Job：在 Runner 上执行的一组 step；不同 job 可并行。
- Artifact：运行结束后保存的报告或构建产物，不等于 Git 提交。

QueueMate 真实代码入口

- .github/workflows/ci.yml
- .github/workflows/full-regression.yml

执行流程

1. push/PR 触发基础 CI
2. checkout
3. 安装 Java/Node/pnpm 并利用缓存
4. Maven test 与 Vite build
5. always 上传报告
6. 全量流程另起 MySQL、后端、Newman、Playwright、JMeter

常见错误与排障

- 把 uses 与 run 混淆
- 服务容器健康前就启动测试
- 失败时不上传日志
- 把 artifact 当部署

本课验收

- 指出 trigger/job/step/runner/service/artifact
- 比较两条工作流的触发范围和成本

第 9.3 课：用真实失败学习 CI 排障

为什么要学

本课问题

CI 故障往往来自干净环境、版本和路径差异；要用日志和最小重现定位，而不是反复盲改。

核心概念（零基础解释）

- 可复现性：锁文件、工具版本和命令在不同环境产生一致结果。
- 工作目录：step 的相对路径起点。
- 失败证据：退出码、日志、报告和上传产物共同描述故障。

QueueMate 真实代码入口

- ci.yml 的 pnpm 11.19.0
- full-regression.yml 的 JMeter 报告路径
- HANDOFF 3.14 与 4

执行流程

1. 定位失败 job/step
2. 读取第一处根因而非最后一行
3. 对比本地与 Runner 版本/路径/环境变量
4. 本地模拟干净运行
5. 最小修复并再次触发
6. 保留报告证明恢复

常见错误与排障

- pnpm 9/11 与锁文件不匹配
- JMeter 在父目录不存在时生成报告失败
- 只看红叉不展开日志

本课验收

- 复盘两个真实故障的现象、根因、修复和证据
- 解释为什么本地通过仍可能远程失败

模块 10：性能工程、排障与工程质量

本模块目标

用数据和分层证据定位问题，诚实识别当前系统边界。

第 10.1 课：性能指标和瓶颈定位

为什么要学

本课问题

单个‘快’或‘慢’没有意义；性能结论必须说明负载、分布、错误率和系统资源。

核心概念（零基础解释）

- 吞吐量：单位时间完成的请求数。
- 延迟分布：平均值可能掩盖慢请求，P90/P95 关注尾部。
- 资源指标：CPU、内存、线程、连接池、锁等待和数据库负载用于解释为什么慢。

QueueMate 真实代码入口

- tests/jmeter 报告与 JTL
- write-summary-zh.ps1
- HikariPool/后端日志
- MySQL 慢查询与执行计划（扩展方向）

执行流程

1. 固定测试环境与数据
2. 逐级增加并发
3. 同时观察延迟/吞吐/错误率
4. 关联后端和数据库资源
5. 定位瓶颈层
6. 修改后用同一基线复测

常见错误与排障

- 只报平均响应时间
- 错误率高时仍讨论延迟优秀
- 用一次本机结果外推生产容量

本课验收

- 解释 12 并发结果能证明什么、不能证明什么
- 为 50/100/300 阶梯并发设计观察指标

第 10.2 课：系统化排障

为什么要学

本课问题

复杂系统故障不能靠猜；需要从用户可见现象逐层寻找第一处异常证据。

核心概念（零基础解释）

- 分层定位：页面 -> Network -> 代理/端口 -> Security/Controller -> Service -> SQL -> 环境。
- 最小复现：减少变量，只保留能稳定触发问题的步骤。
- 相关性与因果：时间上同时发生的日志不一定是根因。

QueueMate 真实代码入口

- 浏览器 DevTools
- http.js 错误归一化
- 后端日志与 GlobalExceptionHandler
- Mapper SQL、CI 日志和测试报告

执行流程

1. 记录准确现象和时间
2. 确认请求是否发出
3. 检查 URL/方法/status/code/body
4. 按 trace 进入后端层
5. 核对 SQL 数据和环境配置
6. 形成根因、修复和回归证据

常见错误与排障

- 页面报错就直接改 CSS
- 看到 401 先查数据库 SQL
- 同时改很多地方导致无法确认根因

本课验收

- 为登录后跳回登录画排障树
- 为预约偶发失败区分 409、500 和网络失败

第 10.3 课：安全、可维护性和当前边界

为什么要学

本课问题

工程能力也包括知道系统尚未解决什么；诚实边界比堆砌高级术语更可信。

核心概念（零基础解释）

- 威胁模型：明确资产、攻击者、入口和可能损失。
- 可维护性：清晰分层、稳定契约、测试和迁移共同降低修改风险。
- 技术债：当前可接受但未来需要偿还的设计限制。

QueueMate 真实代码入口

- HANDOFF 当前卡点与已知边界
- README 非目标
- SecurityConfig、schema.sql、前端 localStorage

执行流程

1. 列出当前资产和入口
2. 对照实现识别已覆盖控制
3. 记录未覆盖风险
4. 按影响与成本排序
5. 用测试保护渐进改进

常见错误与排障

- 虚构 Redis、微服务或真实支付
- 把 localStorage JWT 说成绝对安全
- 忽略限流、Refresh Token、时段重叠、分页等边界

本课验收

- 说出至少六项当前边界
- 为其中一项提出可验证的下一步

模块 11：简历表达与模拟面试

本模块目标

把真实实现、验证证据、取舍和边界组织成可信的技术表达。

第 11.1 课：项目 30 秒、90 秒、3 分钟介绍

为什么要学

本课问题

不同场景要求不同信息密度；介绍应从问题和职责出发，再给方案与证据。

核心概念（零基础解释）

- 电梯陈述：在有限时间内传递背景、职责、方案和结果。
- 信息分层：30 秒给全景，90 秒给两条主链，3 分钟再展开开发和测试。
- 证据化表达：数量、状态和运行结果必须能指向真实代码或报告。

QueueMate 真实代码入口

- README 项目目标与测开亮点
- HANDOFF 验证结果
- 学习总纲的简历就绪检查表

执行流程

1. 一句业务背景
2. 说明三角色与预约/排队
3. 给出 Spring Boot/Vue/MySQL 架构
4. 突出事务并发与自动化
5. 引用 144/45/99/6/12 等真实结果
6. 主动说明模拟支付等边界

常见错误与排障

- 从技术栈清单开始背诵
- 使用无法解释的数字
- 把团队成果和个人理解混淆

本课验收

- 分别录制三个时长版本
- 删去所有无法用仓库证据支持的句子

第 11.2 课：简历项目描述与量化结果

为什么要学

本课问题

简历条目需要短，但不能牺牲真实性；每条都应能被追问到代码、测试或运行证据。

核心概念（零基础解释）

- STAR：情境、任务、行动、结果的组织方式。
- 量化：用真实规模、覆盖范围和结果增强可验证性。
- 边界声明：区分模拟能力、当前实现和生产化方向。

QueueMate 真实代码入口

- README 的最终结果
- 两条 GitHub Actions 工作流
- 测试目录与报告
- 核心 Service/Mapper

执行流程

1. 选 3-5 个最有技术含量的贡献
2. 每条写问题和行动
3. 补可核实结果
4. 准备代码入口
5. 准备取舍和局限追问

常见错误与排障

- 把 ‘参与’ 写成 ‘主导’
- 把生成测试资产写成测试通过
- 只写 ‘熟悉 Spring Boot’ 而没有动作和结果

本课验收

- 为防超卖写一条可核实条目
- 为测试体系写一条不堆工具名的条目

第 11.3 课：高频技术追问与压力追问

为什么要学

本课问题

追问检验的不是背诵，而是能否从设计目标、实现、失败模式和边界持续推理。

核心概念（零基础解释）

- Why-How-Proof-Boundary：为什么做、如何做、证据是什么、尚有什么边界。
- 反事实：如果去掉事务、唯一键或后端鉴权，会发生什么。
- 权衡：说明方案解决的问题、成本和适用规模。

QueueMate 真实代码入口

- 总纲 30 个高频问题
- Auth/Booking/Wallet/Queue 核心代码
- schema.sql 与测试资产

执行流程

1. 先复述问题边界
2. 给结论
3. 用真实链路展开
4. 给失败案例或并发时序
5. 给测试证据
6. 说明替代方案与当前边界

常见错误与排障

- 只说术语不落代码
- 遇到不会的问题虚构实现
- 忽略面试官追问的条件变化

本课验收

- 准备 30 题口头回答
- 对事务、JWT、401/403、防超卖各做两轮反事实追问

第 11.4 课：完整模拟面试与查漏补缺

为什么要学

本课问题

一次连续模拟能暴露知识之间的断点，例如能背 JWT 定义却追不出过滤器链。

核心概念（零基础解释）

- 能力矩阵：概念、代码追踪、实践、表达四个维度分别评分。
- 追问树：从项目介绍沿安全、数据库、并发、前端、测试和 CI 逐层深入。
- 复盘：记录具体卡住的推理步骤，而不是只记‘答得不好’。

QueueMate 真实代码入口

- 学习总纲模块验收表
- student-learning-notes-and-exercises.md
- 本 PDF 的逐课自测清单

执行流程

1. 3 分钟项目介绍
2. 随机选择一条真实链路
3. 连续深挖设计和异常分支
4. 现场读一段代码或设计测试
5. 按四维评分
6. 针对最低维度补课并复测

常见错误与排障

- 只练固定问题顺序
- 老师替学生说完整答案
- 未达到 80% 就继续跳课

本课验收

- 完成 30-45 分钟录音模拟
- 列出三个薄弱点及对应代码和补练任务

第二篇：QueueMate 项目深度知识记录

为什么保留深度篇

逐课篇负责建立学习顺序和概念地图；深度篇完整保留项目开发过程中形成的专题记录，其中包含设计原因、真实测试结果、边界和故障复盘。注意：深度篇按开发时间形成，早期记录中的表数量、测试数量等属于当时阶段；发生差异时，以第一篇的当前事实和当前代码为准。

QueueMate 学习记录

01：项目初始化与本地运行链路

说明
记录日期：2026-07-15
说明
当前阶段：项目骨架、设计文档、Maven、MySQL、Spring Boot 健康接口已完成

1. 本阶段学习目标

这一阶段不是为了快速堆业务代码，而是先建立一条可重复、可验证的本地开发链路：

需求与边界
-> 仓库和模块划分
-> Java/Maven 环境
-> Spring Boot 工程骨架
-> MySQL 表结构与模拟数据
-> 后端启动
-> 健康接口验证
-> 自动化测试与 CI 的扩展入口

完成这条链路后，后续开发注册、登录、预约、排队、钱包等功能时，每个模块都可以在统一工程结构中实现和测试。

2. 当前完成情况

项目	当前结果	验证方式
JDK	Java 21.0.8	java -version
Maven	Apache Maven 3.9.16	mvn -version
后端框架	Spring Boot 3.3.5	应用启动日志
ORM	MyBatis-Plus 3.5.9	Maven 依赖已加载
数据库	MySQL 8.0.44	MySQL 客户端连接成功
数据库初始化	7 张表和模拟数据	SHOW TABLES 与聚合计数
后端测试	1 个测试通过	mvn test 返回 BUILD SUCCESS
Web 服务	8080 端口启动成功	请求 /api/v1/health
IDE	VS Code	Java、Maven、Spring Boot 配置已建立

数据库初始化后的数据量：

表	行数
users	6
venues	3
booking_slots	4
bookings	2

表	行数
queue_tickets	2
wallets	3
wallet_transactions	4

3. 我的设计过程

3.1 先确定项目价值，再选择技术

QueueMate 的目标不是只展示 CRUD，而是展示测开工程能力。因此在设计时优先考虑以下问题：

1. 哪些业务规则适合做接口测试？例如重复预约、余额不足、非法状态流转。
2. 哪些数据竞争适合做性能测试？例如多人同时抢最后几个预约名额。
3. 哪些角色边界适合做权限测试？例如商家不能管理其他商家的地点。
4. 哪些用户流程适合做 UI 自动化？例如登录、选择时段、支付、取消预约。
5. 如何让这些测试能进入 CI，而不是只能在开发者电脑上手工运行？

因此项目从一开始就预留了 Postman、JMeter、Playwright 和 GitHub Actions 目录，而不是等业务完成后再补测试。

3.2 选择单仓库、前后端分离结构

当前采用 monorepo，也就是前端、后端、SQL、测试和文档放在同一个 Git 仓库中：

```
QueueMate/
├── backend/queuemate-server/  Spring Boot 后端
├── frontend/queuemate-web/    Vue 3 前端
├── sql/                      建表与模拟数据
├── tests/                    Postman、JMeter、Playwright
├── docs/                     需求、数据库、API、测试设计
├── knowledge/                分阶段学习记录
├── scripts/                  本地和 CI 辅助脚本
└── .github/workflows/        GitHub Actions
```

这个结构适合个人作品集和中小型项目，因为：

- 一次提交可以同时包含接口、页面和测试。
- CI 可以从同一仓库构建完整系统。
- 文档、SQL 和代码版本保持一致。
- 不需要过早承担微服务的部署、链路追踪和分布式事务成本。

真实业务是否拆成多个仓库或微服务，应由团队规模、发布边界和系统负载决定，而不是因为“微服务更高级”。

3.3 先设计数据和接口，再写业务实现

本项目先完成 requirement、db_design、api_design 和 test_plan，再创建代码骨架。这样做的目的，是先统一以下契约：

- 角色有哪些，分别能操作什么资源。
- 预约和排队分别有哪些状态。
- 容量、余额和状态流转由谁维护。
- API 如何命名、如何鉴权、如何返回错误。
- 测试需要哪些可构造、可查询、可清理的数据。

设计文档不是一次写完后不再变化。开发中发现冲突时，应同时更新文档、代码和测试，使三者保持一致。

3.4 使用最短反馈闭环验证初始化

本阶段采用了由小到大的验证顺序：

```
mvn -version
-> mvn test
-> MySQL 登录
-> 执行 schema.sql
-> 执行 data.sql
-> SHOW TABLES / COUNT(*)
-> mvn spring-boot:run
-> GET /api/v1/health
```

这种顺序有利于快速定位故障。例如 `mvn -version` 失败时没有必要先排查 Spring Boot；MySQL 客户端都无法登录时，也没有必要先排查 MyBatis。

4. Maven 技术点

4.1 Maven 解决什么问题

Maven 主要负责：

- 根据 `pom.xml` 下载和管理 Java 依赖。
- 统一编译、测试、打包和运行命令。
- 通过插件扩展 Spring Boot 启动、测试报告等能力。
- 在本机和 CI 中使用相同构建过程。

本项目的 Maven 坐标是：

```
<groupId>com.queuemate</groupId>
<artifactId>queuemate-server</artifactId>
<version>0.0.1-SNAPSHOT</version>
```

三者共同标识一个构件。`SNAPSHOT` 表示仍处于开发阶段。

4.2 常用生命周期命令

```
mvn clean      # 删除 target 构建产物
mvn compile    # 编译主代码
mvn test       # 编译并执行测试
mvn package    # 测试后生成 jar
mvn spring-boot:run # 通过插件启动 Spring Boot
```

常见原则：CI 至少运行 `mvn test` 或 `mvn verify`，不能只验证能否编译。

4.3 当前主要依赖的职责

依赖	职责
<code>spring-boot-starter-web</code>	REST API、JSON、Tomcat
<code>spring-boot-starter-validation</code>	请求参数校验
<code>spring-boot-starter-security</code>	认证与授权框架
<code>mybatis-plus-spring-boot3-starter</code>	数据访问和常用 CRUD 能力
<code>mysql-connector-j</code>	Java 连接 MySQL 的 JDBC 驱动
<code>jjwt-*</code>	JWT 创建、解析和校验
<code>spring-boot-starter-test</code>	JUnit、Mockito、Spring 测试支持
<code>spring-security-test</code>	模拟用户、角色和鉴权测试

依赖应按实际需要加入。依赖越多，版本冲突、安全漏洞和升级成本越高。

5. Spring Boot 技术点

5.1 启动类

`QueueMateApplication` 是应用入口。`@SpringBootApplication` 组合了自动配置、组件扫描和配置类能力。

启动过程可以理解为：

```
JVM 启动
-> 创建 Spring ApplicationContext
-> 扫描 Bean
-> 应用自动配置
-> 启动内嵌 Tomcat
-> 注册 Controller 路由
-> 监听 8080 端口
```

5.2 配置文件与环境变量

基础配置位于：

```
backend/queuemate-server/src/main/resources/application.yml
```

Spring Boot 支持多种配置来源，常见优先级中，命令行参数和环境变量可以覆盖配置文件。例如不修改仓库文件也能临时覆盖数据库密码：

```
$env:SPRING_DATASOURCE_PASSWORD = '本机数据库密码'
mvn spring-boot:run
```

重要安全原则：

- 真实密码、JWT 密钥和云服务密钥不能提交到 Git。
- 开发、测试、生产使用不同配置。
- 生产环境使用环境变量或专业密钥管理服务。
- 数据库不应长期使用 `root` 账号连接应用，应创建最小权限业务账号。

5.3 健康接口的意义和边界

当前接口：

```
GET /api/v1/health
```

示例响应：

```
{
  "code": "0",
  "message": "success",
  "data": {
    "status": "UP",
    "service": "queuemate-server"
  }
}
```

它证明：

- Java 进程成功启动。
- Tomcat 正在监听 8080。
- Spring MVC 已注册 Controller。
- HTTP 请求和 JSON 序列化正常。

它暂时不能证明：

- 应用已经成功执行 MySQL 查询。
- 所有业务依赖都健康。
- JWT、权限和业务功能已实现。

当前 MySQL 已通过客户端单独验证，但项目还没有 Mapper 查询。因此后续应增加数据库级健康检查或集成测试，真正执行一条 SQL。实际生产中通常使用 Spring Boot Actuator，并谨慎控制健康详情的暴露范围。

6. MySQL 数据库设计技术点

6.1 为什么使用 InnoDB 和 utf8mb4

所有表使用 InnoDB，主要为了事务、行级锁、崩溃恢复和外键支持。字符集使用 utf8mb4，可以完整保存中文和四字节 Unicode 字符。

6.2 七张核心表的职责

表	业务职责
users	用户身份、角色和状态
wallets	用户当前模拟余额
venues	商家拥有的地点
booking_slots	可预约日期、时间、容量和价格
bookings	用户预约单和支付状态
wallet_transactions	充值、支付、退款等余额流水
queue_tickets	现场取号及叫号状态

6.3 主键、唯一约束、外键和索引

这些概念职责不同：

- 主键用于唯一标识一行数据。
- 唯一约束负责阻止业务重复数据。
- 外键保护表之间的引用完整性。
- 普通索引用于提高查询速度，但会增加写入和存储成本。

项目中的典型约束：

约束	解决的问题
users.username 唯一	防止用户名重复
bookings(user_id, active_slot_id) 唯一	防止同一用户重复持有同一时段的 BOOKED 预约，同时允许保留取消历史后重新预约
时段四字段唯一	防止地点出现完全重复的时段
每日排队号唯一	防止同一地点同一天出现重复号码
wallets.user_id 唯一	保证一个用户只有一个钱包
外键	防止预约引用不存在的用户、地点或时段

业务校验不能只写在前端。前端校验改善体验，后端校验保护接口，数据库约束负责最后一道数据一致性防线。

6.4 金额为什么使用 DECIMAL

金额字段使用 `DECIMAL(10,2)`，不能使用 `float` 或 `double`。二进制浮点数可能产生精度误差，而十进制定点数更适合金额计算。

真实支付业务还需要明确：

- 币种。
- 最小金额单位。
- 舍入规则。
- 退款是否允许部分退款。
- 支付和退款幂等号。
- 账务流水是否允许修改。

6.5 钱包余额和流水为什么分表

`wallets.balance` 用于快速读取当前余额，`wallet_transactions` 用于记录余额为什么变化。两者应在同一个数据库事务中更新。

支付时需要同时满足：

```
校验余额充足
-> 原子扣减余额
-> 写入支付流水
-> 更新预约支付状态
-> 整个事务一起提交或一起回滚
```

只修改余额而不写流水，会导致无法审计；只写流水而没有正确更新余额，会导致账实不符。

6.6 预约并发与防超卖

`booking_slots` 同时保存 `capacity` 和 `reserved_count`。不能采用“先查询剩余量，再普通更新”的方式，因为两个并发请求可能同时看到最后一个名额。

推荐使用数据库条件更新：

```
update booking_slots
set reserved_count = reserved_count + 1
where id = ?
and status = 'OPEN'
and reserved_count < capacity;
```

根据受影响行数判断是否抢位成功：

- 受影响 1 行：占位成功。
- 受影响 0 行：时段已满或不可预约。

预约记录创建、容量占用和钱包支付需要进一步设计事务边界。JMeter 后续应验证高并发下 `reserved_count <= capacity`，并检查成功预约数与计数是否一致。

7. SQL 初始化与模拟数据

建表和模拟数据被拆成两个文件：

```
sql/schema.sql # 数据库结构
sql/data.sql   # 本地演示数据
```

这样拆分的好处是结构和数据职责清晰，测试环境可以按需加载不同数据集。

MySQL 客户端中可以执行：

```
source D:/QueueMate/sql/schema.sql;
source D:/QueueMate/sql/data.sql;
```

注意：当前 `schema.sql` 包含 `DROP TABLE IF`

`EXISTS`，适合本地首次初始化或可重建测试环境，不适合直接用于生产升级。真实业务应使用 Flyway 或 Liquibase 管理增量迁移，例如：

```
V1_create_initial_tables.sql
V2_add_booking_payment_columns.sql
V3_add_query_indexes.sql
```

已经上线的表通常通过增量变更演进，不能每次删除重建。

8. VS Code 开发方式

项目已预留以下工作区配置：

- `.vscode/extensions.json`：推荐 Java、Maven、Spring Boot、REST Client 等插件。
- `.vscode/settings.json`：指定 Java 21。
- `.vscode/tasks.json`：提供 `backend: test` 和 `backend: run`。
- `.vscode/launch.json`：提供 Java 调试入口。

命令行和 IDE 并不是两套构建系统。VS Code 最终仍然调用 JDK、Maven 和 Spring Boot，因此要先理解命令行，再使用 IDE 提升效率。

调试相比普通启动的主要价值是：

- 设置断点。
- 查看变量和调用栈。
- 单步执行代码。
- 定位异常发生前的数据状态。

9. 本次遇到的问题与排查方法

9.1 mvn 命令不存在

可能原因：Maven 未安装，或 Maven 的 `bin` 目录没有进入 `PATH`。

排查：

```
mvn -version
$env:MAVEN_HOME
$env:Path
```

修改系统环境变量后，需要重新打开 PowerShell 或 VS Code，旧进程通常不会自动获得新环境变量。

9.2 Maven 无法下载依赖

典型表现是 `Could not transfer artifact`、DNS、连接超时或权限错误。

排查顺序：

1. 检查是否能访问 Maven Central 或团队私服。
2. 检查代理、VPN、防火墙和 `settings.xml`。

- 3. 检查本地仓库中失败下载生成的 `.lastUpdated` 状态。
- 4. 在企业环境中确认私服地址和认证配置。

首次下载依赖较慢是正常的，之后会优先使用本地 Maven 仓库缓存。

9.3 MySQL Access denied

这通常表示账号、密码、主机授权或认证方式不正确，不代表 MySQL 服务没有启动。

应分别检查：

- Windows 的 MySQL 服务是否处于 Running。
- 用户名和密码是否正确。
- 账号是否允许从当前 host 登录。
- 账号是否有目标数据库权限。

9.4 浏览器显示 ERR_CONNECTION_REFUSED

这表示目标地址没有进程接受连接，常见原因：

- Spring Boot 尚未启动。
- 启动过程中报错退出。
- 服务已被手工停止。
- 端口不是 8080。
- 8080 被其他程序占用。

Windows 可以使用以下命令检查端口：

```
Get-NetTCPConnection -LocalPort 8080 -ErrorAction SilentlyContinue
```

它与 HTTP 401、403、404、500 不同：这些 HTTP 状态码说明连接已经到达服务器，只是认证、路由或业务处理出现问题。

9.5 No MyBatis mapper was found

当前还没有创建 Mapper，因此启动日志中的该警告符合现阶段状态。实现用户模块时创建 Mapper 后应再次观察该警告是否消失。

9.6 Spring Security 生成临时密码

因为已引入 Spring Security、但自定义 JWT 登录尚未完成，开发模式下可能看到自动生成的临时密码。后续实现自定义 `SecurityFilterChain`、用户加载和 JWT 过滤器后，应取消默认表单登录行为。

10. 面向真实业务需要继续加强的部分

当前骨架适合学习和本地演示，但距离生产系统还有明显差距：

当前做法	真实业务建议
使用 root 数据库账号	创建最小权限应用账号
本地传入数据库密码	使用环境变量或密钥管理服务
SQL 脚本删除并重建表	使用 Flyway/Liquibase 增量迁移
模拟密码可能使用 <code>{noop}</code>	使用 BCrypt/Argon2 哈希，绝不保存明文
自定义简单健康接口	接入 Actuator，并增加依赖健康检查

当前做法	真实业务建议
手工启动和验证	使用自动化集成测试和 CI
单机 MySQL	根据业务规模设计备份、恢复、高可用和监控
本地模拟钱包	保持账务事务、幂等、审计和对账能力
仅应用日志	增加结构化日志、trace ID、指标和告警

生产化不是一次性把所有技术都加入，而是在明确风险后按阶段增加能力。

11. 当前验证结论

已经可以确认：

- Java、Maven 和 Spring Boot 构建链路正常。
- MySQL 服务可登录，表结构和初始化数据已成功写入。
- Spring Boot 可以监听 8080 并返回健康接口 JSON。
- 项目具备继续开发后端业务模块的基础。

仍需后续验证：

- 通过 MyBatis Mapper 从 Spring Boot 中执行真实 SQL。
- 数据库异常时应用的错误处理和健康状态。
- JWT 登录、角色权限和资源归属校验。
- 预约与钱包事务的一致性。
- 并发预约是否真正不超卖。

12. 下一阶段学习任务

下一阶段建议实现“用户注册、登录与 JWT 鉴权”，学习顺序如下：

1. 用户实体、DTO、Mapper 和 Service 分层。
2. BCrypt 密码哈希与密码校验。
3. 注册参数校验和用户名唯一冲突处理。
4. 登录校验和 JWT 签发。
5. JWT 解析、过期校验和过滤器。
6. **USER**、**MERCHANT**、**ADMIN** 权限规则。
7. Postman 正常、异常、未登录和越权测试。
8. Spring Boot 集成测试连接 MySQL 测试库。

完成后，将学习结果继续记录为：

```
knowledge/02-authentication-and-jwt.md
```

13. 自测问题

如果能独立回答下面的问题，说明已经掌握本阶段核心内容：

1. Maven 的 **compile**、**test**、**package** 分别做什么？
2. 为什么修改 **PATH** 后要重新打开终端？

3. `pom.xml` 中 `runtime` 和 `test scope` 有什么区别?
4. Spring Boot 从启动类到监听 8080 经历了什么?
5. 健康接口成功为什么不一定代表数据库查询成功?
6. 唯一约束、外键和普通索引分别解决什么问题?
7. 为什么金额使用 `DECIMAL` 而不是 `double`?
8. 为什么钱包余额和钱包流水要同时保留?
9. 为什么“先查库存再更新”可能导致超卖?
10. 为什么生产环境不能使用会删除表的初始化脚本?
11. `ERR_CONNECTION_REFUSED` 与 HTTP 500 有什么区别?
12. 为什么真实数据库连接不应该使用 `root` 账号?

QueueMate 学习记录 02：用户注册、登录与 JWT 鉴权

说明

当前阶段：后端认证最小闭环已实现并通过真实 MySQL 冒烟验证

1. 本阶段完成目标

本阶段实现了三个接口：

POST /api/v1/auth/register
POST /api/v1/auth/login
GET /api/v1/auth/me

形成的完整链路是：

注册参数校验
-> 用户名唯一性检查
-> BCrypt 密码哈希
-> 创建 USER
-> 创建零余额钱包
-> 事务提交

登录账号密码校验
-> 检查账号状态
-> 签发 JWT
-> 客户端保存 token

Bearer token
-> JWT 验签和过期检查
-> 查询数据库中的最新用户状态和角色
-> 建立 Spring Security 身份
-> 访问 /auth/me

2. 代码分层

层次	主要文件	职责
Controller	AuthController	接收 HTTP 请求并返回统一响应
DTO	RegisterRequest、LoginRequest 等	定义接口契约和参数校验
Service	AuthService	注册、登录、用户钱包事务
Mapper	UserMapper、WalletMapper	访问 MySQL
Entity	User、Wallet	映射数据库表
Security	JwtAuthenticationFilter	从 Bearer token 恢复身份
Token	JwtTokenService	JWT 签发、验签和解析
Exception	GlobalExceptionHandler	统一业务和系统错误

分层的重点不是文件数量，而是让 HTTP、业务规则、数据访问和安全机制可以分别测试和演进。

3. 注册设计

3.1 为什么只能注册 USER

公开注册接口固定创建 **USER**。不能接受客户端传入 **role**，否则攻击者可以直接注册 **ADMIN** 或 **MERCHANT**。

商家和管理员账号应由后台审批、初始化脚本或管理员接口创建。

3.2 用户名和参数校验

注册请求使用 Jakarta Validation：

- 用户名 3 到 50 个字符。
- 密码 8 到 64 个字符。
- 昵称必填，最多 100 个字符。
- 手机号可选，填写时必须满足当前模拟格式。

参数校验失败统一返回：

```
{
  "code": "PARAM_INVALID",
  "message": "请求参数不合法",
  "data": null
}
```

前端校验用于改善用户体验，后端校验用于保护接口，数据库唯一约束用于处理并发下的最终一致性。

3.3 为什么检查用户名后还要捕获唯一键冲突

只执行“先查询用户名是否存在”仍然存在并发竞态：

```
请求 A：查询，不存在
请求 B：查询，不存在
请求 A：插入成功
请求 B：插入冲突
```

因此业务层先查询用于快速给出友好错误，数据库 `users.username`

唯一约束负责最终防线，代码还要把重复键异常转换为 **409 / USERNAME_EXISTS**。

3.4 用户和钱包为什么使用同一事务

注册成功后，每个普通用户都应该有钱包。因此创建用户和创建钱包必须一起成功或一起失败：

```
@Transactional
public RegisterResponse register(RegisterRequest request) {
    // insert user
    // insert wallet
}
```

如果钱包插入失败，事务会回滚用户插入，避免出现“用户存在但钱包缺失”的半成品数据。

4. 密码安全

4.1 不能保存明文密码

数据库只保存 `password_hash`。新注册用户使用 BCrypt，保存值以 `{bcrypt}` 开头。BCrypt 的特点包括：

- 每次哈希自动加入随机盐。
- 相同密码多次哈希结果通常不同。
- 校验时使用 `matches`，不能解密原密码。
- 计算成本可调，降低暴力破解速度。

4.2 当前密码策略

认证模块初版只实现 8 到 64 个字符的基础长度校验和 BCrypt 哈希，没有实现密码强度分级、泄露密码阻断或复杂字符规则。这是适合第一阶段学习的最小实现，后续再单独设计完整密码策略。

4.3 登录错误信息为什么不区分账号和密码

无论用户不存在还是密码错误，都返回：

```
{
  "code": "AUTH_CREDENTIALS_INVALID",
  "message": "用户名或密码错误",
  "data": null
}
```

这样可以减少攻击者通过接口枚举有效用户名。真实业务还应增加失败次数限制、审计日志、IP/账号维度限流和必要的人机验证。

5. JWT 设计

5.1 token 中包含什么

当前 JWT 包含：

Claim	含义
iss	签发者 QueueMate
sub	用户 ID
iat	签发时间
exp	过期时间
username	用户名
role	USER、MERCHANT 或 ADMIN

JWT 默认有效期是 7200 秒。

JWT 只是签名，不是加密。payload 可以被客户端解码，因此不能放密码、手机号、身份证号等敏感信息。

5.2 签名解决什么问题

服务器使用密钥对 header 和 payload 签名。攻击者可以读取 JWT 内容，但没有密钥就无法合法修改用户 ID、角色或过期时间。

服务端必须验证：

- 签名是否正确。
- iss 是否为预期签发者。
- token 是否过期。
- Claim 类型和值是否可解析。

5.3 为什么每次请求还要查询用户

只相信 token 中的角色会产生状态延迟。例如管理员已经禁用某账号，但旧 token 在两小时内仍可能有效。

当前过滤器根据 token 的用户 ID 查询数据库，并使用数据库中的最新状态和角色建立身份。因此：

- 用户被禁用后，旧 token 会失效。

- 用户角色修改后，权限立即使用新角色。
- token 内仍保留角色 Claim，便于测试和展示，但授权以数据库为准。

代价是每个认证请求增加一次数据库查询。真实高流量系统可以使用短期缓存、token version 或集中式会话方案，在性能和实时失效之间权衡。

5.4 为什么使用无状态 Session

安全配置使用：

```
SessionCreationPolicy.STATELESS
```

服务端不创建传统登录 Session，每次请求都携带 Bearer token。这便于 API、前后端分离和水平扩容，但 token 的存储、刷新、撤销和泄露处理需要单独设计。

6. Spring Security 请求过程

```
HTTP 请求
-> JwtAuthenticationFilter
-> 读取 Authorization: Bearer ...
-> JwtTokenService 验签
-> UserMapper 查询用户
-> 写入 SecurityContext
-> URL 权限规则
-> Controller
```

公开接口：

- [/api/v1/health](#)
- [/api/v1/auth/register](#)
- [/api/v1/auth/login](#)

其他接口默认要求认证。后续地点、钱包、预约模块会继续增加 `hasRole` 或方法级资源权限校验。

7. 401 与 403

状态	含义	示例
401 Unauthorized	没有有效身份	token 缺失、伪造、过期
403 Forbidden	身份有效但没有权限	USER 调用管理员接口

当前统一响应：

```
{
  "code": "AUTH_UNAUTHORIZED",
  "message": "请先登录或提供有效 token",
  "data": null
}

{
  "code": "AUTH_FORBIDDEN",
  "message": "无权访问当前资源",
  "data": null
}
```

8. 配置与密钥

数据库和 JWT 配置支持环境变量覆盖：

```
$env:DB_USERNAME = '本地数据库账号'
$env:DB_PASSWORD = '本地数据库密码'
$env:JWT_SECRET = '至少32字节的随机密钥'
mvn spring-boot:run
```

真实密钥不能提交到 Git。生产环境应使用部署平台 Secret、Vault、云密钥管理服务集中管理，并建立轮换机制。

9. 已完成的测试

自动化测试覆盖：

- 注册创建 **USER**。
- 注册同时创建零余额钱包。
- 新密码使用 BCrypt。
- 重复用户名返回冲突。
- 正确密码登录并返回 JWT。
- 错误密码返回统一认证错误。
- 禁用用户无法登录。
- 当前用户信息从数据库读取。
- JWT 可解析用户 ID、用户名和角色。
- 被篡改 JWT 无法通过验签。
- 非法 JSON 返回 **400/PARAM_INVALID**。

真实 HTTP + MySQL 冒烟覆盖：

- 未登录访问 **/auth/me** 返回 401。
- 伪造 token 返回 401。
- 注册返回新用户 ID 和 **USER**。
- 登录返回 **Bearer** token 和 7200 秒有效期。
- 携带 token 能获取本人信息。
- 数据库密码字段前缀为 **{bcrypt}**。
- 注册后钱包余额为 **0.00**、状态为 **ACTIVE**。
- 冒烟测试账号已清理。

Postman 资产：

- [tests/postman/QueueMate.postman_collection.json](#)
- [tests/postman/QueueMate.local.postman_environment.json](#)

集合会按顺序执行动态用户注册、登录、token 保存、当前用户查询，并验证错误密码和缺失

token。动态注册会在本地数据库留下测试用户，后续可以增加测试数据清理接口或 Newman 执行后的数据库清理脚本。

10. 后续安全参考

- [NIST SP 800-63B](<https://pages.nist.gov/800-63-4/sp800-63b.html>)：后续设计密码长度、阻断列表和规范化策略时参考。
- [OWASP Authentication Cheat Sheet](https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html)：无 MFA 时的密码长度、强度计、常见/泄露密码阻断建议。

- [OWASP Password Storage Cheat Sheet](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html): 密码哈希算法和 BCrypt 72 字节输入限制。

11. 当前边界与后续增强

当前尚未实现:

- Refresh Token。
- 主动登出和 token 黑名单。
- 登录失败次数限制。
- 验证码和风控。
- 密码重置。
- 手机号真实性验证。
- 商家和管理员账号审批。
- token 密钥轮换。
- 权限注解和资源归属校验。

MVP 可以先使用短期 Access Token。等前端登录流程完成后, 再根据实际需要决定是否加入 Refresh Token, 避免一开始过度设计。

12. 自测问题

1. 为什么公开注册接口不能接收客户端传入的角色?
2. 为什么查询用户名不存在后, 插入时仍可能发生唯一键冲突?
3. @Transactional 在注册流程中保护了哪些数据?
4. BCrypt 哈希为什么每次可能不同?
5. 为什么不能从 BCrypt 哈希还原密码?
6. JWT 是加密还是签名? payload 能否被用户看到?
7. iss、sub、iat、exp 分别代表什么?
8. 为什么账号禁用后不能只等待 token 自然过期?
9. 401 和 403 的区别是什么?
10. 为什么 JWT 密钥不能写入 Git?
11. 为什么前端参数校验不能代替后端校验?
12. 真实业务还需要怎样防止暴力登录和用户枚举?

13. 下一阶段建议

下一阶段可以实现地点管理:

```
地点 Entity/Mapper
-> 地点公开查询
-> MERCHANT/ADMIN 创建地点
-> 商家资源归属校验
-> USER/MERCHANT/ADMIN 权限测试
```

对应学习记录建议为:

knowledge/03-venue-management-and-rbac.md

QueueMate 学习记录 03：地点管理与 RBAC

说明

当前阶段：地点管理最小闭环已实现，自动化测试已覆盖公开查询、角色权限和商家资源隔离

1. 本阶段完成目标

本阶段实现五个接口：

```
GET /api/v1/venues
GET /api/v1/venues/{id}
POST /api/v1/venues
PUT /api/v1/venues/{id}
PATCH /api/v1/venues/{id}/status
```

地点列表和详情允许匿名访问。创建、修改和启停要求 **MERCHANT** 或 **ADMIN**，并在 Service 层继续校验资源归属。

2. 为什么角色判断和资源归属要分两层

RBAC 只能回答“这个角色能不能进入某类接口”：

```
USER -> 不可管理地点
MERCHANT -> 可以管理地点
ADMIN -> 可以管理全部地点
```

但 **MERCHANT** 能进入更新接口，不代表可以更新任意地点。因此还需要对象级校验：

```
venue.merchantId == currentUser.id
```

不满足时返回：

```
{
  "code": "RESOURCE_NOT_OWNED",
  "message": "只能操作自己名下的地点",
  "data": null
}
```

Spring Security 的 `@PreAuthorize` 负责角色入口，`VenueService` 负责资源归属。即使 Service 被其他 Controller 或任务调用，也不会绕过核心权限规则。

3. 创建地点时如何确定所属商家

- **MERCHANT** 创建地点时始终使用当前登录用户 ID，不能靠请求体冒充其他商家。
- **ADMIN** 创建地点时必须指定 `merchantId`。
- 管理员指定的账号必须存在、角色为 **MERCHANT** 且状态为 **ACTIVE**。

这样既支持商家自助创建，又支持管理员代建，同时避免客户端直接控制资源归属。

4. 地点名称为什么有两层重复保护

数据库已有唯一索引：

```
uk_venues_name_merchant (name, merchant_id)
```

Service 先查询同名地点，以便返回清晰的 `409 / VENUE_NAME_EXISTS`。并发请求仍可能同时通过查询，所以插入和更新时还要捕获数据库唯一键冲突。数据库约束是最终防线。

5. 地点查询设计

首版不做分页，使用轻量列表查询，支持：

- 按 `category` 精确筛选。
- 按 `status` 精确筛选。
- 使用 `keyword` 模糊匹配名称、描述和模拟地址。
- 按地点 ID 升序返回，保证结果顺序稳定，便于接口测试断言。
- 响应中的 `id` 和 `merchantId` 序列化为字符串，避免 MyBatis-Plus 雪花 ID 超出 JavaScript 安全整数范围后发生精度丢失。

数据量扩大后，再把响应升级为分页结构，避免首版提前引入不必要复杂度。

6. 参数与状态边界

- 地点名称必填，最多 100 个字符。
- 描述最多 500 个字符，模拟地址最多 255 个字符。
- 排队和预约开关必须明确传入布尔值。
- 默认价格不得小于 0，最多 8 位整数和 2 位小数。
- 新地点固定为 `ACTIVE`。
- 状态接口只接受 `ACTIVE` 和 `INACTIVE`。
- 更新基本信息时不能顺便修改归属商家或状态。

7. 401、403 和资源错误

场景	HTTP	错误码
未登录调用地点管理接口	401	<code>AUTH_UNAUTHORIZED</code>
USER 调用地点管理接口	403	<code>AUTH_FORBIDDEN</code>
商家操作其他商家的地点	403	<code>RESOURCE_NOT_OWNED</code>
地点不存在	404	<code>VENUE_NOT_FOUND</code>
同一商家地点名称重复	409	<code>VENUE_NAME_EXISTS</code>

引入方法级权限后发现，`AccessDeniedException` 原先会落入通用异常处理并返回 500。本阶段增加了明确的 403 映射，使 URL 级和方法级拒绝保持统一 JSON 响应。

8. 已完成的测试

`VenueServiceTest` 共 11 个测试，覆盖：

- 地点列表映射。
- 商家创建时强制绑定本人。
- 管理员为有效商家创建地点。
- 管理员缺少商家 ID 被拒绝。
- USER 不能创建地点。
- 商家修改自己的地点。

- 商家不能修改其他商家的地点。
- 管理员可停用任意地点。
- 地点不存在返回 404。
- 同一商家同名地点冲突。
- 可选文本标准化。

`VenueControllerSecurityTest` 共 5 个测试，覆盖匿名公开查询、匿名创建 401、USER 创建 403、MERCHANT 创建成功和请求参数错误。

当前全量 Maven 测试为 27 个，全部通过。

真实 HTTP + MySQL 回归还验证了公开列表和详情、非法筛选、USER 禁止创建、商家归属强制绑定、同名冲突、跨商家越权和管理员停用。回归生成的临时地点已按精确 ID 删除，并确认测试数据残留数量为 0。

9. Postman 资产

地点集合新增：

- 公开地点列表和详情。
- USER 创建地点被拒绝。
- 商家登录并保存 token。
- 商家创建、更新和停用自己名下的地点。

创建请求使用动态名称并保存地点 ID。执行真实集合后需要清理生成的地点数据。当前只验证了集合 JSON 结构，未执行 Newman，因此不能标记 Newman 已通过。

10. 下一阶段建议

下一阶段实现固定预约时段：

```
BookingSlot Entity/Mapper
-> 公开查询地点时段
-> 商家/管理员创建时段
-> 地点归属复用
-> 容量、时间范围和重复时段校验
-> 为并发预约防超卖准备原子更新
```

QueueMate 学习记录 04：固定预约时段

说明

当前阶段：固定预约时段的查询、创建和状态管理已实现，尚未进入用户预约与并发容量扣减

1. 本阶段完成目标

本阶段实现三个接口：

```
GET /api/v1/venues/{venueId}/slots
POST /api/v1/venues/{venueId}/slots
PATCH /api/v1/venues/{venueId}/slots/{slotId}/status
```

公开用户可以查询时段。地点所属商家和管理员可以创建、打开或关闭时段。

2. 为什么先做固定时段，再做预约

预约防超卖依赖一个稳定的容量载体。`booking_slots` 先定义：

- 哪一天可以预约。
- 开始和结束时间。
- 总容量 `capacity`。
- 已预约数量 `reservedCount`。
- 免费或收费价格。
- 当前是否开放。

下一阶段提交预约时，只需围绕某个确定的 `slotId` 做原子容量更新，而不是临时计算商家营业时间。

3. 查询规则

时段查询允许匿名访问，支持：

- `dateFrom`：起始日期，包含边界。
- `dateTo`：结束日期，包含边界。
- `status`：OPEN 或 CLOSED。

`dateFrom` 晚于 `dateTo` 时返回 400 / PARAM_INVALID。结果按日期、开始时间和时段 ID 排序，便于页面展示和接口断言。

响应额外计算：

```
availableCapacity = capacity - reservedCount
```

这个字段只是当前余量展示。真正预约时仍必须使用数据库条件更新，不能先读余量再普通写入。

4. 创建规则

创建时段前依次校验：

```
地点存在
-> 当前用户是地点所属商家或管理员
-> 地点状态为 ACTIVE
-> bookingEnabled = true
```

- > 日期不早于今天
- > 开始时间早于结束时间
- > 容量和价格合法
- > 精确重复时段不存在
- > 插入 OPEN / reservedCount=0 时段

商家资源归属复用 `VenueService.requireOwnerOrAdmin`，避免预约、排队等后续模块各写一套不同规则。

5. 重复时段保护

数据库唯一索引为：

```
uk_booking_slots_unique (venue_id, slot_date, start_time, end_time)
```

Service 先查询，用于返回友好错误；插入时继续捕获唯一键冲突，处理两个并发请求同时通过查询的竞态。两种路径都统一返回：

```
409 / BOOKING_SLOT_EXISTS
```

首版只禁止完全相同的时间范围，不额外禁止相互重叠的时段。这与当前数据库约束一致，避免在场地可能拥有多个并行资源时误判。

6. 时段状态

- 新建时段固定为 OPEN。
- OPEN 可以切换为 CLOSED，阻止后续新预约。
- 关闭时不清空或减少已有 reservedCount。
- CLOSED 可以重新打开，但地点仍必须启用且支持预约。
- 路径中的时段不属于指定地点时，返回 404 / BOOKING_SLOT_NOT_FOUND，不暴露其他地点的时段细节。

7. ID 精度

MyBatis-Plus 默认生成的雪花 ID 可能超出 JavaScript 安全整数范围。因此时段响应中的以下字段序列化为字符串：

- id
- venueId
- createdBy

日期和时间继续使用 ISO 字符串，价格使用两位小数语义。

8. 已完成测试

`BookingSlotServiceTest` 共 13 个测试，覆盖：

- 公开查询和可用余量。
- 反向日期范围。
- 正常创建。
- 停用地点和预约开关。
- 非法时间和过去日期。
- 业务查询重复与数据库并发重复。
- 关闭和重新打开。
- 时段与地点不匹配。

- 跨商家资源归属失败。

`BookingSlotControllerSecurityTest` 共 7 个测试，覆盖公开查询、非法状态参数、匿名 401、USER 403、商家创建、参数校验和关闭时段。

加上认证与地点模块，当前 Maven 全量共 47 个测试。

真实 HTTP + MySQL 回归覆盖了公开查询、反向日期范围、USER 禁止创建、商家创建、重复时段、跨店越权、管理员关闭、商家重开、路径地点不匹配、非法时间和预约开关。回归产生的临时时段已按精确 ID 删除，并确认残留数量为 0。

9. 当前边界

本阶段没有实现：

- 用户提交预约。
- `reservedCount` 增减。
- 并发防超卖。
- 重复预约限制。
- 预约扣款或退款。
- 已有时段的容量、时间和价格编辑。

这些能力将在后续预约事务中实现，避免把时段配置与用户预约状态混在同一轮。

10. 下一阶段建议

下一阶段实现用户预约最小闭环：

```
POST /bookings
-> USER 身份限制
-> 校验地点和时段可用
-> 原子 reservedCount + 1
-> 唯一约束阻止重复预约
-> 创建免费预约记录
-> 并发测试验证不超卖
```

收费预约和钱包扣款建议紧接其后，并在同一事务内扩展。

QueueMate 免费预约与并发防超卖

说明

本文记录免费预约阶段的历史实现。收费预约已在后续钱包与消费码模块中上线，当前行为以 [knowledge/07-wallet-voucher-implementation.md](#) 为准。

1. 本轮目标

本轮完成用户预约的第一个可运行闭环：

- **USER** 创建免费预约。
- 查询当前用户自己的预约。
- 用户取消本人预约，**ADMIN** 可取消任意预约。
- 阻止重复预约和容量超卖。
- 取消时只回补一次名额。
- 收费时段暂不创建预约，等待钱包支付模块。

2. 为什么先做免费预约

收费预约还需要钱包余额条件扣减、支付流水、退款流水和幂等保护。如果在钱包模块之前直接把收费预约标为成功，会形成错误业务状态。

因此本轮采用明确的阶段边界：

- **price = 0**：创建 **BOOKED** 预约，**payStatus=NOT_REQUIRED**，**paidAmount=0**。
- **price > 0**：返回 **409/BOOKING_PAYMENT_REQUIRED**，不占名额、不写预约。

3. 创建预约的事务顺序

创建预约在一个事务中完成：

1. 校验调用者是 **USER**。
2. 查询时段和地点。
3. 校验地点启用、允许预约、时段开放且未开始。
4. 校验当前为免费时段。
5. 预检查同一用户是否已预约该时段。
6. 用数据库条件更新原子增加 **reserved_count**。
7. 插入 **bookings** 记录。

核心容量更新同时校验时段和地点状态：

```
update booking_slots bs
join venues v on v.id = bs.venue_id
set bs.reserved_count = bs.reserved_count + 1
where bs.id = ?
and bs.status = 'OPEN'
and bs.reserved_count < bs.capacity
and v.status = 'ACTIVE'
and v.booking_enabled = 1;
```

受影响行数为 1 才代表抢到名额。并发请求会在 MySQL 中竞争同一行，达到容量后其余请求无法再满足 `reserved_count < capacity`。

容量更新和预约插入处于同一事务。如果插入失败，前面的容量增加也会回滚。

4. 重复预约的双重保护

重复预约不能只依赖“先查再插”，因为两个并发请求可能同时查询到不存在。

本轮使用两层保护：

- Service 预检查，尽早返回可读的 `BOOKING_DUPLICATE`。
- 数据库生成列 `active_slot_id` 只在状态为 `BOOKED` 时取 `slot_id`，唯一键 `uk_bookings_user_active_slot (user_id, active_slot_id)` 处理并发竞态。

取消后生成列变为 `NULL`。MySQL 唯一索引允许多条

`NULL`，因此历史取消记录可以保留，用户也能重新预约同一时段；新的有效预约仍会再次占用唯一键。

数据库抛出的重复键异常也映射为 `409/BOOKING_DUPLICATE`，并由事务回滚已经增加的名额。

5. 取消预约的一致性

取消只允许从 `BOOKED` 进入 `CANCELLED`：

```
update bookings
set status = 'CANCELLED',
    cancel_reason = ?,
    cancelled_at = ?
where id = ?
and status = 'BOOKED';
```

只有受影响行数为 1 时才继续回补容量：

```
update booking_slots
set reserved_count = reserved_count - 1
where id = ?
and reserved_count > 0;
```

这两个更新位于同一事务。两个请求并发取消同一预约时，只有一个请求能把 `BOOKED` 改成 `CANCELLED`，因此名额只会回补一次。

6. 权限和接口边界

- `POST /api/v1/bookings`：仅 `USER`。
- `GET /api/v1/bookings/my`：仅 `USER`，只查询本人。
- `PATCH /api/v1/bookings/{id}/cancel`：预约本人或 `ADMIN`。
- `MERCHANT` 不能代表用户创建或取消预约。
- 所有 64 位 ID 在 JSON 响应中序列化为字符串。

7. 测试结果

自动化回归：

- `BookingServiceTest`：15 个。
- `BookingControllerSecurityTest`：10 个。

- 后端全量：72 个测试全部通过。

真实 HTTP + MySQL 并发验证：

- 临时时段容量：3。
- 并发用户数：12。
- 成功预约：3。
- `BOOKING_SLOT_FULL`：9。
- 最终 `reserved_count`：3。
- 最终有效预约数：3。
- 重复用户时段组合：0。
- 同一预约并发取消：1 成功、1 个 `BOOKING_STATUS_INVALID`。
- 取消后重新预约成功，容量和有效预约数继续一致。

验证完成后，临时预约、时段、钱包和用户数据均已清理。

8. 下一步

下一模块接入站内钱包：

1. 钱包查询和模拟充值。
2. 收费预约的余额条件扣减。
3. 支付流水与预约创建同事务。
4. 已支付预约取消退款。
5. 防重复扣款、防重复退款和并发余额一致性。

QueueMate 付费预约与消费码设计

1. 设计结论

QueueMate 的收费预约不收取额外“预约手续费”。时段的 `price` 表示用户需要预付的消费金额：

- 奶茶店可把预付金额用于到店消费抵扣。
- 自习室可把消费码兑换为对应时长的使用权。
- 羽毛球场可把消费码作为该场次费用的支付凭证。

付费成功后生成消费码。消费码不是另一份余额，而是“这笔预约已经预付”的核销凭证。

2. 为什么消费凭证独立建模

如果只在 `bookings` 表增加一个字符串字段，很难清楚记录核销状态、有效期、核销人、退款作废和过期时间。

目标设计新增与收费预约一对一的 `BookingVoucher`：

Booking 1 --- 0..1 BookingVoucher

- 免费预约没有消费凭证。
- 收费预约支付成功后必须有且只有一张消费凭证。
- 凭证金额必须等于预约的 `paidAmount`。

3. 状态模型

预约状态：

```
BOOKED -> FULFILLED
|
+----> CANCELLED
|
+----> NO_SHOW
```

支付状态：

```
PAID -> REFUNDED
```

消费凭证状态：

```
AVAILABLE -> REDEEMED
|
+----> VOID
|
+----> EXPIRED
```

三个状态必须匹配：

场景	Booking	PayStatus	Voucher
付费预约成功	BOOKED	PAID	AVAILABLE
商家核销成功	FULFILLED	PAID	REDEEMED
用户取消并退款	CANCELLED	REFUNDED	VOID
到期未使用	NO_SHOW	PAID	EXPIRED

4. 收费预约事务

收费预约必须在同一数据库事务中完成：

1. 校验用户、地点、时段和重复预约。
2. 原子占用时段名额。
3. 使用余额条件更新完成扣款。
4. 写入 `PAYMENT` 钱包流水。
5. 写入 `BOOKED/PAID` 预约。
6. 写入 `AVAILABLE` 消费凭证。

任一步失败时全部回滚，不能出现以下中间状态：

- 扣款成功但没有预约。
- 预约成功但没有消费码。
- 生成消费码但没有扣款。
- 占用名额后预约记录插入失败。

5. 消费码生成

首版消费码建议使用 10 到 16 位大写字母和数字的安全随机值，例如：

```
QM7K9P2X5A8
```

约束：

- 全局唯一。
- 不由预约 ID、用户 ID、手机号或时间戳直接推导。
- 数据库唯一索引处理极小概率的随机碰撞。
- 用户本人可以查看完整码。
- 商家预约列表不批量返回完整码。
- 日志最多记录后四位，禁止打印完整码。

当前是本地作品集项目，可在数据库保存消费码明文以支持用户重复查看。若未来升级为真实业务，应改为加密存储、签名令牌或二维码方案。

6. 核销规则

目标接口：

```
POST /api/v1/venues/{venueId}/booking-vouchers/redeem
```

核销条件：

- 调用者是地点所属 `MERCHANT` 或 `ADMIN`。
- 消费码属于路径地点。
- 预约状态是 `BOOKED`。
- 支付状态是 `PAID`。
- 凭证状态是 `AVAILABLE`。
- 当前时间位于预约开始前 30 分钟至时段结束时间。

核销成功时，在同一事务内：

1. 条件更新凭证为 **REDEEMED**。
2. 记录 **redeemedBy** 和 **redeemedAt**。
3. 条件更新预约为 **FULFILLED**。

条件更新保证并发提交同一消费码时只有一个请求成功。

7. 取消与退款

时段开始前且凭证仍为 **AVAILABLE** 的收费预约可以全额退款。时段开始后首版不允许用户自助取消。

退款事务包含：

1. 条件更新预约 **BOOKED -> CANCELLED**。
2. 回补时段名额。
3. 钱包余额退款。
4. 写入 **REFUND** 流水。
5. 条件更新消费凭证 **AVAILABLE -> VOID**。

消费凭证已经 **REDEEMED** 时禁止取消和退款。

退款和核销可能并发发生，因此两条流程都必须以 **AVAILABLE** 为前置状态。谁先成功改变凭证状态，另一条流程就必须失败并回滚。

8. 到期未使用

时段结束后仍为 **AVAILABLE** 的消费凭证可通过定时任务或管理操作变为 **EXPIRED**，对应预约变为 **NO_SHOW**。

首版默认过期不自动退款，用于模拟场地预订和爽约场景。未来可以按地点配置退款比例或宽限期。

9. 关键测试

- 收费预约成功后扣款、流水、预约和消费凭证同时存在。
- 余额不足时四类业务数据都不新增。
- 同一预约不会重复扣款或生成多张消费凭证。
- 商家不能跨店核销。
- 同一消费码并发核销只有一次成功。
- 已核销消费码不能退款。
- 退款后消费码不能核销。
- 核销与退款并发时只能有一条状态路径成功。
- 核销时间窗口边界正确。
- 日志和错误响应不泄露完整消费码。

10. 实现顺序

1. 钱包查询、充值和流水。
2. 新增消费凭证表及枚举。
3. 接入收费预约原子事务。

4. 实现用户消费码查询。
5. 实现商家核销。
6. 实现取消退款与消费码作废。
7. 实现过期和 `NO_SHOW`。
8. 完成并发验证、Postman 和 JMeter 资产。

QueueMate 钱包、付费预约与消费码实现

说明

本文记录钱包与消费码模块当时的实现基线。自动过期、管理员钱包接口、排队和统计已在 [knowledge/08-backend-completion.md](#) 中完成。

1. 本轮完成内容

- 钱包余额查询、模拟充值和本人流水查询。
- 收费预约的预付消费扣款。
- 支付成功后生成一对一消费凭证。
- 商家核销消费码并完成预约。
- 时段开始前取消收费预约时全额退款并作废消费码。
- 并发付费预约、并发核销和退款/核销互斥保护。

2. 钱包一致性

同一用户的钱包余额变更前先执行：

```
select *
from wallets
where user_id = ?
for update;
```

行锁保证同一钱包的充值、支付和退款串行执行，因此流水中的 `balance_before` 与 `balance_after` 可以准确对应实际余额。

支付仍使用余额条件更新：

```
update wallets
set balance = balance - ?
where id = ?
and status = 'ACTIVE'
and balance >= ?;
```

受影响行数不是 1 时，返回余额不足或钱包不可用，事务中不会留下预约、流水或消费码。

3. 收费预约事务

收费预约的一个事务包含：

1. 校验地点、时段、重复预约和容量。
2. 原子增加 `reserved_count`。
3. 锁定钱包并扣减预付金额。
4. 写入 `PAYMENT` 钱包流水。
5. 写入 `BOOKED/PAID` 预约。
6. 写入 `AVAILABLE` 消费凭证。

`wallet_transactions(biz_type, biz_no, type)` 防止同一预约重复扣款，`booking_vouchers.booking_id` 防止生成多张消费凭证。

4. 消费码

消费码格式为：

```
QM + 10 位安全随机字符
```

字符集去除了容易混淆的 `I/O/0/1`。数据库唯一键处理随机碰撞，最多尝试生成 5 次。

用户通过“我的预约”获得完整消费码。核销接口和业务异常不输出完整码，代码中也没有记录完整消费码的日志。

5. 核销与退款竞态

核销和退款都会先对同一条 `booking_vouchers` 记录执行 `for update`。

核销路径：

```
AVAILABLE -> REDEEMED  
BOOKED -> FULFILLED
```

退款路径：

```
AVAILABLE -> VOID  
BOOKED/PAID -> CANCELLED/REFUNDED  
钱包退款 + REFUND 流水 + 名额回补
```

两条事务谁先锁定并改变凭证状态，另一条就会在拿到锁后发现状态不再是 `AVAILABLE`，从而失败并回滚。这避免了“已经消费又退款”的双重成功。

6. 权限边界

- 钱包接口仅允许 `USER`。
- 创建预约仅允许 `USER`。
- 用户只能查看自己的完整消费码。
- 核销仅允许地点所属 `MERCHANT` 或 `ADMIN`。
- 商家提交其他地点的消费码返回 `RESOURCE_NOT_OWNED`。
- 用户取消本人预约，管理员可按相同退款窗口取消预约。

7. 自动化结果

本轮新增：

- `WalletServiceTest`: 5 个。
- `WalletControllerSecurityTest`: 4 个。
- `BookingVoucherServiceTest`: 7 个。
- `BookingVoucherControllerSecurityTest`: 4 个。

后端全量测试：

```
Tests run: 94, Failures: 0, Errors: 0, Skipped: 0
```

8. 真实并发验证

测试时段容量为 3，6 个用户各充值 50，并发预约价格 20 的同一时段：

- 3 个请求成功，3 个请求返回 409。

- 成功用户余额为 30，失败用户余额仍为 50。
- `reserved_count=3`。
- 支付流水数量为 3。
- 消费凭证数量为 3。

对同一消费码并发提交两次核销：

- 1 个请求返回 200。
- 1 个请求返回 409。
- 数据库只有一次核销结果。

临时用户、钱包、流水、预约、消费码和时段均已清理。

9. 当前边界

- 当时尚缺的消费码自动过期、`NO_SHOW` 和管理员钱包接口已在后续后端收官迭代完成。
- Postman 集合已经更新，但尚未通过 Newman 正式运行。
- 后续模块的现场排队状态机和繁忙统计已经完成。

QueueMate 后端收官实现

1. 本轮范围

本轮把需求文档中的剩余后端能力一次性补齐：

- 现场取号、本人号码和公开排队进度。
- 商家叫号、完成、过号和管理员全局操作。
- 并发安全的地点每日递增号码。
- 管理员钱包流水和余额调整。
- 消费码自动过期与预约 `NO_SHOW`。
- 预约与排队合并的繁忙时段统计。

前端、Newman/JMeter/Playwright 正式执行和 GitHub Actions 不属于本轮后端实现范围。

2. 并发取号序列

直接执行下面的逻辑存在竞态：

```
select max(queue_no) + 1 from queue_tickets;
```

多个事务可能同时读到相同最大值。因此新增 `queue_daily_sequences`，主键为：

```
(venue_id, queue_date)
```

每次取号在事务中执行：

```
insert into queue_daily_sequences (venue_id, queue_date, last_no)
values (?, ?, last_insert_id(1))
on duplicate key update
last_no = last_insert_id(last_no + 1);
```

随后在同一数据库连接读取：

```
select last_insert_id();
```

MySQL 会对主键冲突记录加行锁，因此同一地点同一天的并发请求会串行获得不同序号。

3. 有效号码防重

`queue_tickets.active_flag` 是生成列：

```
WAITING / CALLED -> 1
COMPLETED / MISSED -> null
```

唯一索引：

```
(venue_id, queue_date, user_id, active_flag)
```

这允许用户在号码进入终态后重新取号，同时禁止同一用户并发获得两个有效号码。

4. 状态机

排队状态只允许：

```
WAITING -> CALLED
CALLED -> COMPLETED
CALLED -> MISSED
```

状态修改使用带旧状态条件的更新：

```
update queue_tickets
set status = ?
where id = ?
and status = ?;
```

并发叫号或重复完成时只有一个请求能更新一行，另一个请求返回 `QUEUE_STATUS_INVALID`。

5. 自动过期

定时任务按固定间隔执行两个事务内更新：

1. 把 `valid_until` 已过期的 `AVAILABLE` 凭证更新为 `EXPIRED`。
2. 把这些凭证对应的 `BOOKED/PAID` 预约更新为 `NO_SHOW`。

核销、退款和过期都以 `AVAILABLE` 为条件，因此同一凭证只能有一条路径成功。

6. 管理员余额调整

管理员调整金额支持正负数：

- 正数：增加余额。
- 负数：扣减余额。
- 0：拒绝。
- 扣减后小于 0：拒绝。

调整前先锁定钱包行，并写入 `ADJUSTMENT` 流水。流水金额保存绝对值，方向由 `balance_before` 和 `balance_after` 表达。

7. 繁忙时段统计

统计不维护冗余汇总表，而是实时聚合：

- 预约：按 `booking_slots.start_time` 的小时统计。
- 排队：按 `queue_tickets.taken_at` 的小时统计。
- 热度：`bookingCount + queueCount`。

商家只能查看自己地点，管理员可查看全部地点。查询日期范围最多 366 天。

8. 验证结果

自动化测试：

```
Tests run: 126, Failures: 0, Errors: 0, Skipped: 0
```

真实 MySQL 并发取号：

- 6 个用户同时请求同一地点。
- 6 个请求全部成功。
- 获得连续且唯一的 3-8 号。
- 数据库序列表 `last_no=8`。

同一号码并发叫号：

```
200, 409
```

自动过期验证：

```
BOOKED / AVAILABLE  
-> NO_SHOW / EXPIRED
```

管理员余额增加后再扣减，最终余额恢复，调整流水完整。所有临时用户、号码、序列、流水、预约、消费码、时段和数据库账号均已清理。

9. 当前结论

需求文档范围内的后端功能已经完成。下一阶段可以进入 Vue3 前端、正式接口/性能/UI 自动化执行和 GitHub Actions。

Vue3 前端与设计系统实现记录

本轮目标

在后端接口保持不变的前提下，为空的 `frontend/queuemate-web` 初始化 Vue3 应用，并先完成普通用户最短业务闭环。设计先于页面实现，统一规则保存在 `design-system/MASTER.md`。

技术实现

- Vue 3 + Vite
- Vue Router 路由与角色守卫
- Element Plus 组件和图标
- Axios 请求、Bearer Token 注入与统一错误处理
- CSS 语义 tokens 和移动优先响应式布局
- 路由级动态导入与 Element Plus 组件按需注册

用户端页面包括：

```
登录
-> 地点列表
-> 地点详情
-> 选择时段并预约 / 现场取号
-> 我的预约 / 钱包 / 我的排队
```

设计决策

QueueMate 采用“城市服务台”方向：服务蓝负责导航与主要操作，状态色只用于明确反馈，场所、预约、消费码和排队号码使用克制的票据式信息结构。页面不使用渐变、玻璃效果、装饰动画和大面积胶囊圆角。

设计系统统一定义：

- 颜色与对比度
- 中文字体栈与数据字体
- 4px 间距体系
- 4/8/12px 圆角
- 阴影和边框
- 按钮、表单、卡片、状态、票据和反馈规则
- 375/768/1024/1440px 响应式检查点
- 键盘焦点、44px 触控目标与减少动画设置

验证

生产构建：

```
cd D:\QueueMate\frontend\queuemate-web
pnpm install
pnpm build
```

最后一次构建成功。登录、地点列表、地点详情、我的预约、钱包和我的排队页面使用 Chrome DevTools 设备模拟，在 375px 与 1440px 共 12 个页面/视口组合中完成检查：没有横向溢出，没有浏览器运行时错误。

视觉 QA 使用符合当前 DTO

的临时本地响应，不写入产品代码或数据库。本会话后端未启动，因此仍需在后续联调中使用真实 Spring Boot + MySQL 验证登录、预约、充值、取消和取号请求。

下一步

1. 真实后端联调用户端写操作。
2. 实现商家叫号、消费码核销和繁忙统计页面。
3. 实现管理员地点与钱包调整页面。
4. 将本轮浏览器检查转为仓库内正式 Playwright 用例。

Vue3 全角色前端完成记录

1. 本轮目标

在不修改后端业务逻辑和接口的前提下，把 QueueMate 从用户端第一版扩展为访客、普通用户、商家和管理员都可使用的完整 Web 前端。

2. 功能覆盖

访客与普通用户

- 新增普通用户注册页，复用后端用户名、密码、显示名称和手机号校验边界。
- 登录后按角色进入对应工作区，原目标路由继续保留。
- 地点类别改为后端真实枚举 `TEA_SHOP / STUDY_ROOM / BADMINTON_COURT`。
- 我的预约支持状态筛选、取消原因、消费码有效期和复制。
- 我的排队支持按日期、地点和状态查询历史号码。
- 钱包流水根据 `balanceBefore` 与 `balanceAfter` 判断收支方向，不能直接把始终为正数的 `amount` 当成入账。

商家与管理员场所运营

- 地点创建、修改、启用和停用。
- 预约时段创建、日期/状态筛选、打开和关闭。
- 查看当天或指定日期排队进度，对 `WAITING` 叫号，对 `CALLED` 完成或过号。
- 输入消费码核销收费预约。
- 按日期范围查看预约与排队的小时热度。
- 商家前端只展示自己名下地点，后端继续执行最终资源归属校验。

管理员工具

- 管理全部地点并复用场所运营工作台。
- 查询全局钱包流水，按用户 ID 和流水类型筛选。
- 对指定用户执行正负余额调整。
- 按预约 ID 执行管理员取消。

3. 后端接口边界如何反映在页面

前端不能伪造后端没有提供的数据：

- 后端没有商家用户列表接口，因此管理员创建地点时需要手动输入商家用户 ID。
- 后端没有全局预约列表接口，因此管理员取消预约时需要手动输入预约 ID。
- 钱包流水响应没有用户 ID，管理员需要先按用户 ID 发起筛选，页面明确提示这一点。

这些是 API 能力边界，不应该在前端通过静态假数据伪装成完整查询功能。

4. 设计系统落实

- 继续使用 `design-system/MASTER.md` 的城市服务台方向。
- 注册页使用“创建账号 -> 选择地点 -> 预约或取号”的服务流程作为唯一视觉签名。

- 商家运营页按真实业务链组织时段、排队、核销和统计，而不是拆成无关联的模板卡片。
- 使用服务蓝、冷灰画布、矩形票据、虚线分隔和等宽数字；没有渐变、玻璃效果、过度圆角或装饰动画。
- 增加跳到主要内容的键盘入口、持久表单标签、加载按钮、错误反馈和 `prefers-reduced-motion` 支持。

5. 验证结果

生产构建：

```
Vite production build: SUCCESS
```

真实浏览器只读联调：

```
15 page/viewport/role combinations passed  
Horizontal overflow: 0  
Browser runtime errors: 0  
HTTP 4xx/5xx: 0
```

检查覆盖注册桌面/手机/横屏减少动画、用户预约/钱包/排队、商家地点/运营工作台、管理员地点/钱包/预约处理。浏览器使用当前运行中的 Spring Boot + MySQL 完成真实登录和只读请求；没有执行会污染数据库的写操作。

6. 下一步

把临时浏览器检查转为仓库内正式 Playwright 用例，加入测试数据创建、写操作断言和按外键顺序清理，然后接入 GitHub Actions。

Postman/Newman 可重复全量回归

1. 为什么不能直接运行旧集合

旧集合包含注册、充值、创建地点和时段、预约、退款、取号、叫号与管理员余额调整。Postman 脚本只能调用 HTTP API，而业务 API 刻意没有提供删除用户、地点、时段和钱包流水的能力，因此直接全量运行会持续污染本地开发库。

2. 本轮方案

- 每次注册请求生成唯一 `runId`。
- 动态用户名使用 `qm_<runId>`。
- 测试地点使用 `Postman Venue <runId>`。
- 管理员调整备注包含同一 `runId`。
- 集合保存后端返回的地点、时段、预约和排队 ID，避免把 64 位 ID 转为 JavaScript Number。
- 集合最后调用专用清理端点，按外键顺序删除本轮数据。
- 清理响应返回 `remainingArtifacts`，Postman 断言必须为 0。

3. 清理端点的安全边界

清理端点需要同时满足：

1. Spring profile 为 `e2e`。
2. `queuemate.test-support.enabled=true`。
3. 请求身份为 `ADMIN`。

清理服务不会按模糊前缀批量删除。用户还必须匹配固定测试昵称和手机号；地点必须匹配运行标记与本地商家夹具；时段必须属于固定测试地点、由测试商家近期创建；调整流水备注必须包含本轮 `runId`。任一现存资源不满足约束时返回 `409/TEST_DATA_UNSAFE` 并回滚事务。

4. 中断与补偿

Runner 和 Newman 均不使用 `bail`，保证普通断言失败后仍会走到 Teardown。如果运行在管理员余额增加后中断，清理服务会计算本轮标记调整流水的净变化，先反向恢复余额，再删除流水。排队号码删除后，会根据剩余号码重新计算每日序列，不会盲目删除其他用户的序列状态。

如果进程或网络在 Teardown 前完全中断，保留当前 Postman 变量，恢复 E2E 服务后单独执行管理员登录与 `Cleanup current Postman run`。

5. Newman 命令

```
cd D:\QueueMate\tests\postman
pnpm install
pnpm test:report
```

`package.json` 固定 Newman 6.2.2，`pnpm-lock.yaml` 固定传递依赖。JSON 报告是运行产物，不提交 Git。

6. 首次正式结果

```
Requests: 45, failed: 0
Assertions: 99, failed: 0
```

Average response time: 39 ms
Run-scoped remaining artifacts: 0

这说明“集合文件存在”“JSON

可解析”和“全量回归已通过”是三个不同层次的结论；只有保留真实执行命令、断言汇总和清理证据，才能标记 Newman 通过。

附录：关键事实速查

类别	当前项目事实
运行端口	Vite 5173; Spring Boot 8080; MySQL 3306
核心数据表	users、user_roles、merchant_applications、wallets、venues、booking_slots、bookings、booking_vouchers、wallet_transactions、queue_daily_sequences、queue_tickets
成功响应	ApiResponse: code='0', message='success', data=业务数据
并发预约证据	12 用户抢容量 3: 3 个 201, 9 个 409/BOOKING_SLOT_FULL; reservedCount=3
后端测试	144 个自动化测试
接口回归	Newman 45 个请求、99 个断言、0 失败, 清理 remainingArtifacts=0
UI 回归	Playwright 6 条核心业务链路, 使用独立浏览器上下文和 finally 清理
CI	基础构建与全量回归两层 GitHub Actions; 报告 artifact 保留 14 天
明确边界	模拟钱包; 无 Refresh Token/黑名单/登录限流; 时段不防重叠且核心字段暂不可编辑; 我的预约未分页

学习完成标准

能够用自己的话解释概念, 沿真实文件追完整链路, 独立完成小范围验证, 并诚实说明设计取舍和边界。任何一课建议达到 80% 再继续下一课。